

IMPLEMENTATION NOTE

Traffic Light Control System

Team QNX

Document information	
Course	EEET2588 EEET2687 - Real-Time Systems Engineering
Team	Team QNX
Members	Tran Dong Nghi - s3914633 Le Hung - s4061665 Hoang Minh Thang - s3999925
Lecturer	Dr Linh Tran
Semester	Semester 2, 2026
Due date	18/09/2026
Repository / commit	https://github.com/lucietran03/traffic-control-system/

Declaration and Statement of Authorship

1. I/we have not impersonated, or allowed myself to be impersonated by, any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
4. Where this work is being submitted for individual assessment, I declare that it is my original work and that no part has been contributed by, produced by or in conjunction with another student.
5. I/we give permission for my assessment response to be reproduced, communicated compared and archived for the purposes of detecting plagiarism.
6. I/we give permission for a copy of my assessment to be retained by the university for review and comparison, including review by external examiners.
7. I/we understand that:
 - Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.
 - Plagiarism includes the act of assisting or allowing another person to plagiarise or to copy my work.

I/we agree and acknowledge that:

1. I/we have read and understood the Declaration and Statement of Authorship above.
2. If I/we do not agree to the Declaration and Statement of Authorship in this context and a signature is not included below, the assessment outcome is not valid for assessment purposes and will not be included in my final result for this course.

Table of Contents

<i>Declaration and Statement of Authorship</i>	<i>2</i>
<i>1. Implementation Overview.....</i>	<i>7</i>
1.1 Purpose and Implemented Scope.....	7
1.2 Deployment Summary.....	8
1.3 Relationship to the Initial Design.....	8
<i>2. Build, Deployment, and Operation</i>	<i>9</i>
2.1 Prerequisites	9
2.2 Build Procedure.....	9
2.3 Deployment and Startup Sequence	10
2.4 Operator and Sensor Controls.....	11
<i>3. Implemented System Architecture</i>	<i>12</i>
3.1 Runtime Architecture.....	12
3.2 Major Components and Processes	13
3.3 Concurrency, Timing, and Synchronization	14
<i>4. Inter-Process Communication</i>	<i>15</i>
4.1 Communication Topology	15
4.1.1 Name Attachment and Discovery.....	15
4.1.2 Dynamic Node Resolution (TRAFFIC_NODE_MAP).....	16
4.1.3 Communication Roles and Authority	16
4.1.4 Dedicated IPC Client Thread.....	17
4.1.5 Asynchronous Pulse Timers	17
4.1.6 Heartbeat Cadence and State Resynchronization	17
4.2 Message Pathways	18
4.3 Message Validation and Failure Handling	20
4.3.1 Wire-Format Contract and Alignment	20
4.3.2 Demultiplexing and Verb-Based Dispatch.....	20
4.3.3 Autonomous FSM Safety Checks.....	20
4.3.4 Negative Acknowledgement (NACK) Taxonomy.....	21
4.3.5 One-Second Response Deadline (PA-12).....	21
4.3.6 Autonomous Railway Protection Under Faults (RC-10).....	21
<i>5. Source-Code Organization</i>	<i>22</i>
5.1 Submitted QNX Workspace Structure	22

5.2 Project, File, and Module Responsibilities.....	23
6. Key Implementation Decisions	24
7. Implemented Features and Operational Evidence.....	26
7.1 Normal traffic and pedestrian operation	26
7.1.1 Deterministic Peak Fixed-Timing Baseline (UC-01, TL-01)	26
7.1.2 Sensor-Actuated Vehicle Detection & Anti-Starvation Cap (UC-01, TL-02)	27
7.1.3 Pedestrian Pushbutton Latching & Clearance Cycle (UC-02)	29
7.2 Railway Protection and Closure Traffic	30
7.2.1 The 45-Second Approach Warning & Hardware Gate Interlock (UC-04, RC-01–RC-04)	30
7.2.2 Adjacent Intersection Direct Traffic Suppression (CC-02, RLx → Lx)	31
7.2.3 Dual-Track Second Train Handling (UC-04 Alt 8.1)	33
7.2.4 Post-Crossing Congestion Management & Queue Drain Phase (CC-01, UC-05)	33
7.2.5 Boom Gate Failure Latching & Safe Recovery (UC-06, RC-06, RC-09)	34
7.3 Central Monitoring, Commands, and Local Autonomy	35
7.3.1 Central Telemetry Aggregation & Real-Time HMI (UC-09)	35
7.3.2 Priority Vehicle & Dignitary Clear-Route Override (UC-08, PA-11)	36
7.3.3 The 4-Tier Supervisory Safety Hierarchy & Interlock Rejections (NACKs)	37
7.3.4 Arterial Green-Wave Coordination & Mode Management (UC-03, UC-07)	38
7.3.5 Communication Loss (LINK_DEGRADED_LOCAL) & Instant State Sync (UC-10, PA-07, PA-08)	39
8. Verification and Test Results	40
8.1 Test Environment and Method.....	40
8.2 Requirements and Scenario Coverage	41
8.3 Timing and Real-Time Behavior.....	42
8.4 Faults, Limitations, and Unresolved Results	43
9. Conclusion	44
References.....	44

LIST OF FIGURES

Figure 1. Traffic System Layout [1].....	7
Figure 2. Implementation controller responsibilities, local inputs and communication path.....	8
Figure 3. As-built task and deployment architecture	13
Figure 4. Implemented Qnet IPC message flows	16
Figure 5. Heartbeat delivery and Central-link state update	18
Figure 6. QNX Momentics workspace organization and standard directory responsibility	22
Figure 7. Full 90-second deterministic peak fixed-timing phase cycle (PEAK_FIXED)	27
Figure 8. Sensor-actuated vehicle detection, dynamic green extension, and anti-starvation cap (OFF_PEAK_SENSOR) (1).....	28
Figure 9. Sensor-actuated vehicle detection, dynamic green extension, and anti-starvation cap (OFF_PEAK_SENSOR) (2).....	28
Figure 10. Pedestrian pushbutton latching, protected walk interval, and flashing clearance cycle	29
Figure 11. 45-second approach warning, boom gate closure, and confirmation-before-proceed train signal interlock (1).....	30
Figure 12. 45-second approach warning, boom gate closure, and confirmation-before-proceed train signal interlock (2).....	31
Figure 13. Adjacent intersection direct traffic suppression (holding connector red toward tracks)	32
Figure 14. Dual-track second train occupancy extension without barrier reopening	33
Figure 15. Post-crossing congestion queue detection and dynamic green flush drain phase	34
Figure 16. Boom gate obstruction failure latching, train stop signal, and recovery verification rejection....	35
Figure 17. Central 1 Hz real-time network telemetry monitoring matrix	36
Figure 18. Priority vehicle / dignitary clear-route override dispatch and lifecycle management	37
Figure 19. Local safety arbitration: VIP override deferred during pedestrian walk and rejected during railway preemption.....	38
Figure 20. Coordinated arterial green-wave timing profile broadcast and safe-boundary mode switching	38
Figure 21. Heartbeat loss fail-safe degradation (LINK_DEGRADED_LOCAL) and instant reconnection resynchronization.....	39

LIST OF TABLES

Table 1. Implemented deployment summary	8
Table 2. Material design-to-implementation changes.....	8
Table 3. Central Controller Operator Commands.....	11
Table 4. Intersection Sensor Controls	11
Table 5. Railway Sensor and Demo Controls	12
Table 6. Controls common to all controllers	12
Table 7. Implemented processes and responsibilities.....	13
Table 8. Concurrency and synchronization summary	15
Table 9. Implemented inter-process communication pathways	18
Table 10. Shared IPC Contracts & Utilities (Replicated across all projects)	23
Table 11. Central Controller Project (<code>central_controller/</code>)	23
Table 12. Intersection Controller Project (<code>intersection_controller/</code>)	24
Table 13. Railway Controller Project (<code>railway_controller/</code>)	24
Table 14. Key Implementation Decisions	24
Table 15. Representative test results	41
Table 16. Representative timing results	42
Table 17. Known issues and limitations	43

1. Implementation Overview

This section provides a high-level overview of the implemented QNX-based traffic control system, outlining its functional scope and distributed deployment architecture. It also details material engineering deviations from the initial design to accurately reflect the final proof-of-concept.

1.1 Purpose and Implemented Scope

The implementation is a **QNX-based proof of concept** for the traffic network defined in the project brief [1]. It covers **six signalized intersections (I1–I6)** across **five roads (R1–R5)** and **three railway crossings** on a double-track railway corridor.

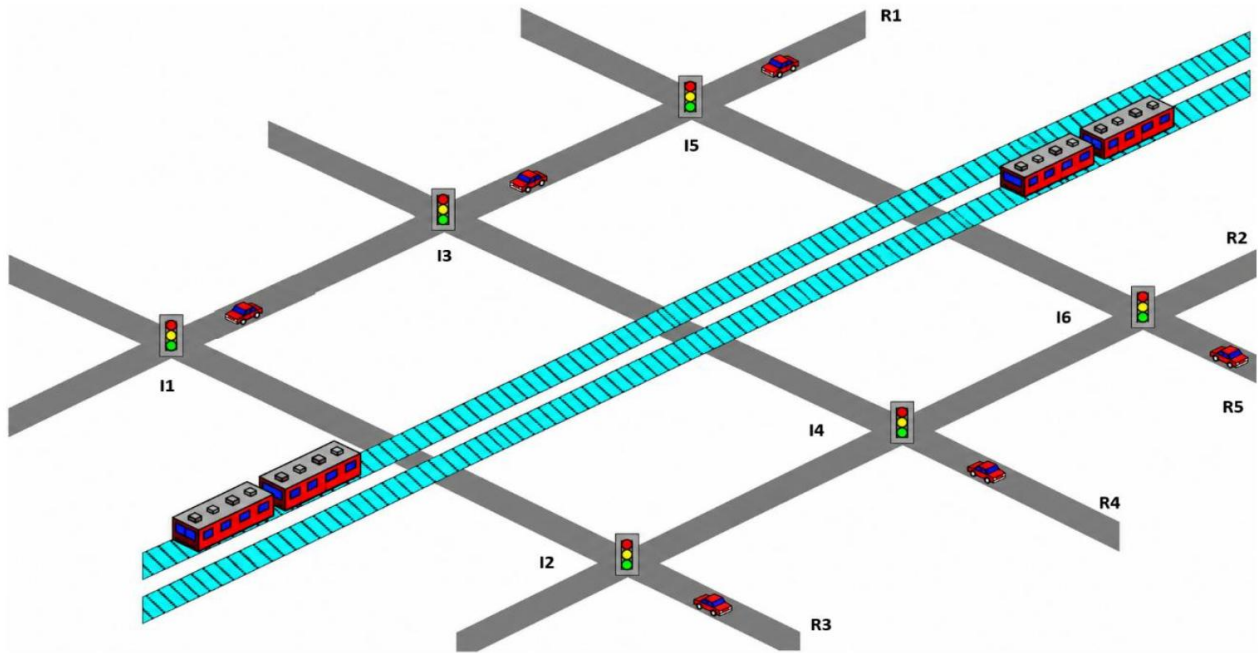


Figure 1. Traffic System Layout [1]

The implemented system comprises **ten controller instances** following the controller allocation in the Initial Design Report [2]: one Central controller (C1), six intersection controllers (L1–L6), and three railway controllers (RL1–RL3). The implemented scope is defined as follows:

- **Intersection control - L1–L6:** Each Lx owns its vehicle and pedestrian signal outputs. It provides fixed-timing and sensor-driven operation, latches pedestrian-button requests, and applies local railway preemption when the associated crossing is unavailable.
- **Railway protection - RL1–RL3:** Each RLx owns one simulated railway crossing, including gate motion and confirmation, warning flashers, and train signals. It reports crossing-state changes directly to the associated Lx controllers and to C1.
- **Central supervision - C1:** C1 monitors the nine local controllers, records system events, and sends operating-mode, timing-profile, override, and fault-clear requests. Local controllers validate these requests and retain authority over their outputs; C1 does not directly actuate road signals or railway equipment [1].
- **Proof-of-concept interface:** Keyboard input simulates vehicle presence, pedestrian push-buttons, queue warnings, train approaches, and gate faults. Controller terminals display simulated road signals, pedestrian indications, gates, warning flashers, train signals, and system status. No physical sensors or actuators are connected.

- **Local autonomy:** Lx and RLx continue their local state machines when communication with C1 is unavailable. Lx can select Peak or Off-Peak operation using its local clock, while railway protection and local fault handling remain under the authority of the respective field controller.

Figure 2 summarizes the implemented controller responsibilities, local input sources, output ownership, and supervisory communication paths.

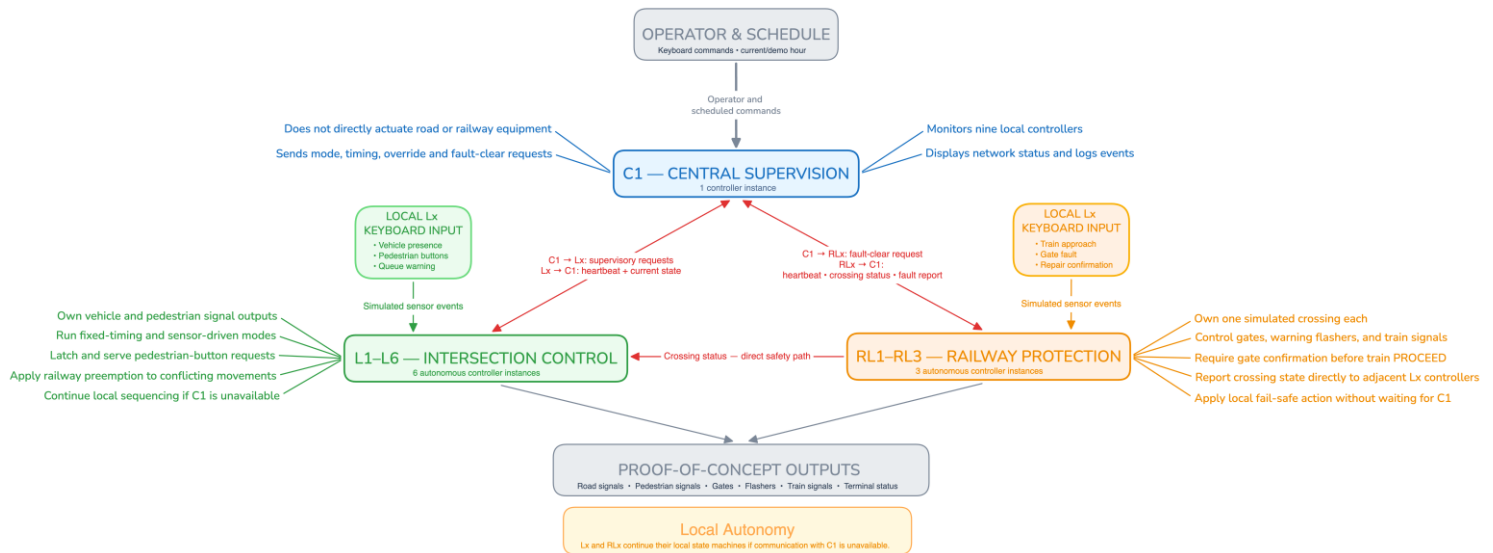


Figure 2. Implementation controller responsibilities, local inputs and communication path

Material differences between the implemented proof of concept and the Initial Design Report [2] are recorded in **Section 1.3**.

1.2 Deployment Summary

Table 1. Implemented deployment summary

Item	Implemented configuration
QNX deployment	Ten QNX virtual machines connected through Qnet; one controller instance per VM in the demonstrated deployment.
Logical controllers	C1 (Central), L1–L6 (intersections), and RL1–RL3 (railway crossings).
QNX projects	Central_Controller, Intersection_Controller, and Railway_Controller. The intersection and railway programs take a numeric controller ID when launched.
Build environment	QNX Momentics and QNX SDP. Each C project has its own Makefile, src/, and includes/ directories.
Operator and sensor input	Dedicated threads read keyboard input from the respective controller terminals.
Display	Controllers print their state and simulated outputs to terminals. C1 periodically renders a network-status view.
Event log	C1 appends events to <code>central_log.txt</code> in its current working directory. If the file cannot be opened, logging continues standard output.
Inter-node communication	QNX message passing over Qnet. <code>TRAFFIC_NODE_MAP</code> supplies the Qnet node name for each controller ID.

1.3 Relationship to the Initial Design

Table 2 records the material differences between the Initial Design Report [2] and the implemented proof of concept. The resulting limitations are considered when interpreting the demonstration and test results.

Table 2. Material design-to-implementation changes

Design item	Initial design	Implemented result	Effect on the implementation
Deployment allocation	Ten logical controllers distributed across two physical computing machines, without requiring one computer per controller [2].	Ten QNX VMs, each running one controller instance.	The demonstration gives every logical controller its own VM; this changes deployment allocation, not the controller-to-controller message paths.

Hardware interface	Sensor-input and signal-driver tasks represent vehicle, pedestrian, train, and road/rail equipment interfaces [2].	Keyboard-reader threads provide simulated inputs; terminal messages represent signal and equipment outputs.	Control sequences can be demonstrated, but electrical inputs, actuator behaviour, and physical sensor reliability are outside this proof of concept.
Boom-gate model (RC-01)	Two independently represented boom gates and position sensors at each crossing [2].	Each RLx uses one aggregate simulated gate-motion state with open/closed confirmation.	The system exercises confirmation before train <code>PROCEED</code> but cannot test disagreement between two separate gates or sensors.
Watchdog allocation	A separate watchdog process supervises each local controller process and can force safe outputs if that process stops [2].	Lx and RLx use watchdog threads inside their controller processes.	A thread can detect a stalled control loop while the process remains alive but cannot supervise or take over after the whole process stops.
Intersection status reporting	Local status reporting accompanies the heartbeat mechanism [2]. The brief also requires a status message whenever an intersection light changes state [1].	Lx sends a 1 s heartbeat containing a status summary; it does not send <code>MSG_STATUS</code> on every signal transition.	C1 learns a changed signal state on the next successfully delivered heartbeat, rather than immediately on each transition.

2. Build, Deployment, and Operation

This section outlines the procedures required to compile the QNX Momentics workspace, deploy the executable binaries across the *Qnet* network, and sequence the system startup. It also defines the terminal-based operator and sensor controls used to interact with the simulated hardware.

2.1 Prerequisites

- **Development environment:** QNX Software Development Platform (SDP) 7.1, QNX Momentics IDE 8.0.3, and the QNX C toolchain (`qcc`). The source is organized as three QNX projects: `Central_Controller`, `Intersection_Controller`, and `Railway_Controller`.
- **Target environment:** Ten QNX VMs, with one controller instance per VM. Their network adapters must be on a network reachable by *Qnet*, *Qnet* must be running on every VM, and each VM must have a distinct *Qnet* node name. Peer node names should be visible under `/net` before launching the controllers.
- **Cross-node addressing:** Export `TRAFFIC_NODE_MAP` in each controller's launch shell. It maps controller suffixes (`c1`, `l1–l6`, `r11–r13`) to the corresponding *Qnet* node names. The variable can be left unset only when the communicating controllers run on the same QNX node.
- **Console access:** Each controller needs a terminal for operator or simulated sensor input and for its status and output messages. Keyboard-driven sensor simulation is permitted by the project brief [1].
- **Libraries:** The reviewed source uses QNX Neutrino and POSIX system APIs; it does not require a separate third-party application library.

2.2 Build Procedure

Step 1: Import Projects: Launch the QNX Momentics IDE. Navigate to *File > Import...* and choose *Projects from Folder or Archive* under *General* category. Browse and select the submitted .zip file (`TeamQNX_A2_ProjectSourceCode.zip`). In the import window, ensure you select just the 3 Eclipse projects (`central_controller`, `intersection_controller`, and `railway_controller`) from the list. Tick *Add project to working sets* for easier workspace management, and then click *Finish*.

- **Note:** Ensure you do not have any existing projects with these exact same names in your workspace before importing. Once finished, you should see all 3 projects clearly listed in your

Project Explorer. If the C/C++ Indexer database becomes corrupted, restart the QNX Momentics IDE.

Step 2: Compile: Highlight all three projects, right-click, and select Build Project to compile the distributed node binaries.

Step 3: Verify Output: Confirm the build finishes without errors. The resulting executable binaries will be automatically generated and located within the Binaries or build directories inside each respective project folder.

Step 4: Clean and Rebuild: If diagnosing a failed build or clearing stale artifacts, right-click the projects, select Clean Project to remove compiled object files, and perform the build step again.

2.3 Deployment and Startup Sequence

Step 1: Network & Environment Initialization: First ensure all VM runs on your computer connected via *qnet*. The system utilizes a custom startup.sh script (located in *output/build/startup.sh*) to automate the cluster configuration. To allow the assessor to reproduce this environment correctly, the following custom blocks must be inserted into specific locations within the standard QNX *startup.sh* script:

1. **Insert before console initialization** (just above on *-d -t /dev/con1 ksh -l*):

```
export
TRAFFIC_NODE_MAP="c1=c1_vm,l1=l1_vm,l2=l2_vm,l3=l3_vm,l4=l4_vm,l5=l5_vm,l6=l6_vm
,r11=r11_vm,r12=r12_vm,r13=r13_vm"
```

2. **Insert at the very end of the file** (immediately following */proc/boot/post_startup.sh*):

```
HOST=$(hostname)
IP=""

case "$HOST" in
  l1_vm) IP="169.254.216.81" ;;
  l2_vm) IP="169.254.216.82" ;;
  l3_vm) IP="169.254.216.83" ;;
  l4_vm) IP="169.254.216.84" ;;
  l5_vm) IP="169.254.216.85" ;;
  l6_vm) IP="169.254.216.86" ;;
  r11_vm) IP="169.254.216.91" ;;
  r12_vm) IP="169.254.216.92" ;;
  r13_vm) IP="169.254.216.93" ;;
  c1_vm) IP="169.254.216.100" ;;
  *) echo "Unknown hostname $HOST; skipping static IP assignment" ;;
esac

if [ -n "$IP" ]; then
  ifconfig wm0 $IP netmask 255.255.0.0 up
fi
```

- o **Note for Setup:** Ensure the target QNX Virtual Machines are configured with the exact hostnames expected by the script (*yourhostname*). Upon booting the VMs with this modified OS image, the script will execute automatically. Before proceeding, verify that *Qnet* is active and that each target node can successfully see its peers in the */net* directory namespace.

Step 2: Binary Deployment: Transfer the compiled executable binaries from the host build environment to the */tmp* directory on the respective target QNX Virtual Machines. Ensure the files have the necessary execution permissions (*chmod +x*).

Step 3: Execution Order: Navigate to the */tmp* directory and launch the compiled binaries in the following sequence to prevent initial communication drops:

- **Central Node:** Start the Central controller first to establish the `traffic/c1` attach-point and commence server loop monitoring.
- **Railway Nodes:** Start the Railway controllers by passing the target ID (1-3) as a command-line argument (e.g., `./railway_controller 1` to initialize `RL1`).
- **Intersection Nodes:** Start the Intersection controllers by passing the target ID (1-6) as a command-line argument (e.g., `./intersection_controller 1` to initialize `L1`).

Step 4: Verification: Observe the Central controller's terminal. The 1 Hz `c_hmi_render()` table should be updated to display all 9 connected controllers. Confirm that their availability status automatically transitions from `UNAVAILABLE` to `AVAILABLE` (and internal links transition from `DEGRADED_LOCAL` to `CENTRAL_CONNECTED`) as initial 1-second heartbeats are acknowledged.

Step 5: Shutdown Procedure: To safely halt the system, input the `q` command in each terminal to cleanly break the blocking stdin sensor and operator reader threads. Follow this with a standard OS interrupt (`Ctrl + C`) to terminate the underlying infinite server loops and exit the processes.

2.4 Operator and Sensor Controls

Each controller uses a dedicated keyboard-input thread so that blocking terminal input does not stop its IPC server or state-machine processing. Central commands prompt for any required parameters after the command key is entered. **Keys are case-sensitive:** for Lx inputs, lowercase keys assert a sensor condition and the corresponding uppercase keys clear it.

Tables 3–5 list the controls specific to C1, Lx, and RLx. **Table 6** lists the controls available on all three controller types.

Table 3. Central Controller Operator Commands

Key	Action	Observable result
m	Request <code>PEAK_FIXED</code> or <code>OFF_PEAK_SENSOR</code> for a selected Lx.	Lx validates the request. A change to a different mode is applied at a safe phase boundary.
t	Send a timing profile to arterial chain R1 (L1/L3/L5) or R2 (L2/L4/L6).	C1 sends each target Lx its assigned profile parameters for local validation.
o	Request a bounded clear-route override for a selected Lx and movement.	Lx accepts, defers during pedestrian clearance, or rejects the request according to its current state.
r	Renew an active override for a selected Lx.	If accepted, Lx restarts the remaining-duration timer using the entered duration. Entering 0 reuses the stored duration; it does not add time to the remaining value.
c	Cancel an active or pending override for a selected Lx.	If accepted, Lx terminates the override and follows its safe clearance sequence before normal operation resumes.
f	Request fault clearance for a selected Lx or RLx.	Lx clears its fault flags and ACKs, then resumes according to its last crossing state. RLx <code>ACKs</code> only if it is in <code>FAULT</code> and its gate is confirmed <code>OPEN</code> ; otherwise, it <code>NACKs</code> .
d	Force a simulated hour from 0 to 23.	C1 selects the scheduled mode for that hour and broadcasts <code>SET_MODE</code> to all six Lx controllers.
a	Resume automatic mode selection using the real clock.	C1 removes the simulated-hour override. On the next 1 s check, it broadcasts a mode change if the real hour implies a different mode.

Table 4. Intersection Sensor Controls

Key	Simulated input	Observable result
a / A	Assert / clear arterial vehicle presence.	Demand affects phase selection or green extension in <code>OFF_PEAK_SENSOR</code> ; it does not change fixed green durations in <code>PEAK_FIXED</code> .
c / C	Assert / clear connector vehicle presence.	Demand affects connector service in <code>OFF_PEAK_SENSOR</code> ; it does not change fixed green durations in <code>PEAK_FIXED</code> .
1 / 2 / 3 / 4	Press the pedestrian button for side 0 / 1 / 2 / 3.	The request is latched and served with a compatible vehicle phase through <code>WALK</code> , <code>FLASHING_DONT_WALK</code> , and <code>DONT_WALK</code> .

w / W	Assert / clear the advance queue warning.	If the warning is active when the associated crossing reopens, Lx arms a connector-drain episode. Connector green can extend in 4 s increments while the warning remains active, subject to the configured cap.
--------------	---	---

Table 5. Railway Sensor and Demo Controls

Key	Simulated input	Observable result
0 / 1	Report a train approaching from direction 0 / 1.	RLx starts its crossing-warning and gate-protection sequence for the selected direction.
x	Make the next simulated gate movement fail confirmation.	The next gate movement does not produce the expected confirmation. If gate closure remains unconfirmed until the configured deadline, RLx enters FAULT, keeps all train signals at STOP, and reports the fault to C1.
r	Force simulated gate-open confirmation for the fault-clear demonstration.	Sets the simulated gate feedback to confirmed OPEN, allowing a subsequent railway fault-clear request to satisfy RLx's gate-safety condition.
f	Invoke the demo-only local fault-clear path.	Calls the RLx fault-clear handler without receiving a request from C1. RLx returns ACK only when it is in FAULT and its gate is confirmed OPEN; otherwise, it returns NACK.
h / ?	Display help.	Prints the commands available in the current controller terminal.
q	Stop keyboard input.	Stops only that controller's operator or sensor-input thread. Its IPC server and state-machine processing continue running.

The **r** and **f** controls are demonstration aids for exercising the railway recovery path. In normal distributed operation, C1 initiates fault clearance using its **f** operator command.

Table 6. Controls common to all controllers

Key	Simulated input	Observable result
h / ?	Display terminal help.	Prints the keyboard commands available for the current controller type.
q	Stop keyboard input.	Terminates only the current controller's operator or sensor-input thread. The IPC server, timers, watchdog, and local state-machine processing continue running.

3. Implemented System Architecture

3.1 Runtime Architecture

The deployed system consists of **ten QNX VMs**, with one controller instance per VM:

- **C1:** one `Central_Controller` instance.
- **L1–L6:** six `Intersection_Controller` instances.
- **RL1–RL3:** three `Railway_Controller` instances.

The program entry points are `src/c_main.c`, `src/lx_main.c`, and `src/rlx_main.c`. Each Lx and RLx executable accepts a numeric controller ID at launch. `TRAFFIC_NODE_MAP` maps the ten controller IDs to their Qnet node names. This deployment implements the controller allocation defined in the Initial Design Report [2].

- **Central supervision - C1:** Receives heartbeats from all nine local controllers and crossing-status and fault reports from RL1–RL3. It displays network status, records events, and sends mode, timing-profile, override, and fault-clear requests. C1 does not directly actuate road or railway outputs [1].
- **Intersection control - L1–L6:** Each Lx owns its vehicle and pedestrian outputs and continues local sequencing without C1. It starts in `LINK_DEGRADED_LOCAL`, changes to `LINK_CENTRAL_CONNECTED` after a valid heartbeat ACK, and returns to `LINK_DEGRADED_LOCAL` after three heartbeat attempts are recorded without a valid ACK. While degraded, Lx uses its local clock for Peak or Off-Peak mode selection.
- **Railway protection - RL1–RL3:** Each RLx owns its simulated gates, warning flashers, and train signals. Its crossing-protection FSM and local fault handling continue independently of C1 availability.

- **Direct railway safety path:** RL1 reports to L1/L2, RL2 to L3/L4, and RL3 to L5/L6. Each RLx also reports to C1. This direct RLx-to-Lx path allows local railway preemption without a C1 relay. Lx cannot command railway equipment.

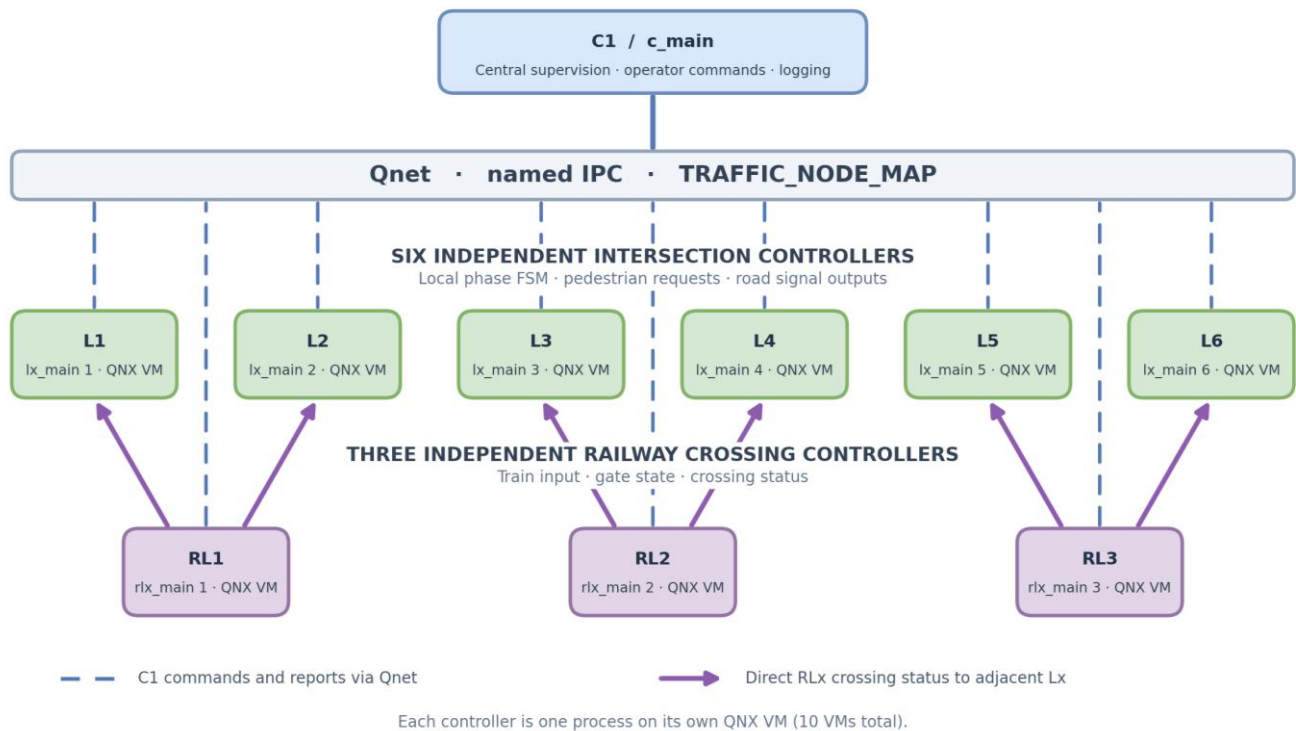


Figure 3. As-built task and deployment architecture

3.2 Major Components and Processes

The deployed system runs **ten controller instances across ten QNX VMs**: one C1, six Lx, and three RLx. The instances are built from three independently buildable QNX Momentics projects:

- Central_Controller
- Intersection_Controller
- Railway_Controller

Each project contains its controller-specific source and header files together with its own copies of the shared IPC definitions and Qnet utilities. The program entry points are `src/c_main.c`, `src/lx_main.c`, and `src/rlx_main.c`.

Table 7. Implemented processes and responsibilities

QNX project / controller	Responsibility	Key inputs	Key outputs	Source location
Central_Controller (C1)	Maintains the network-status view; checks the availability of the nine local controllers; selects scheduled modes; handles operator commands; and records system events.	Operator keyboard commands; heartbeats from Lx and RLx; crossing-status and fault reports from RLx; 1 s timer pulse.	Mode, timing-profile, override, and fault-clear requests; replies to incoming messages; terminal network-status display; central_log.txt.	src/:c_main.c, c_mode_eng.c, c_operator.c, c_comm.c, c_server.c, c_hmi.c, c_logger.c, c_watchdog_mon.c.
Intersection_Controller (L1–L6)	Runs each intersection's local FSM for vehicle phases, pedestrian indications, railway preemption, overrides, and local	Simulated vehicle, pedestrian-button, and queue-warning inputs; C1 requests; crossing status from the adjacent RLx;	Simulated road and pedestrian indications; heartbeat containing the current status	src/:lx_main.c, lx_fsm.c, lx_timer.c, lx_sensor.c, lx_signal.c, lx_comm.c,

	fault handling.	100 ms phase and 1 s heartbeat pulses.	summary; replies to C1 and RLx requests.	lx_watchdog.c.
Railway_Controller (RL1–RL3)	Runs each crossing's local FSM for train approaches, gate motion and confirmation, warning flashers, train signals, occupancy timing, and faults.	Simulated train-approach, gate-fault, and repair-confirmation inputs; C1 fault-clear request; 1 s crossing-FSM and 1 s heartbeat pulses.	Simulated gate, flasher, and train-signal indications; crossing status to C1 and two adjacent Lx controllers; heartbeat and fault reports to C1; replies to incoming requests.	src/:rlx_main.c, rlx_fsm.c, rlx_timer.c, rlx_sensor.c, rlx_gate.c, rlx_signal.c, rlx_comm.c, rlx_watchdog.c.

Each QNX project also contains its own copies of:

- includes/ipc_msg.h
- includes/sys_types.h
- includes/qnet_utils.h
- src/qnet_utils.c

This duplication allows the three submitted projects to be imported and built independently without requiring a separate shared-library project.

The **sensor readers and watchdogs are threads within their Lx or RLx controller process**, not separate executables. Their timing and shared-state access are described in **Section 3.3**.

3.3 Concurrency, Timing, and Synchronization

Thread ownership

- **All controllers:** The main thread runs the IPC server loop. It receives incoming requests and timer pulses, dispatches them sequentially, and replies to synchronous requests.
- **Dedicated IPC client thread:** Each controller uses a separate client thread for outgoing `name_open()` and blocking `MsgSend()` operations. Application threads place outgoing requests into a 16-entry FIFO queue instead of performing cross-node sends directly.
- **C1:** A dedicated operator thread reads keyboard commands. It shares the network-status and scheduling state with the main server thread.
- **Lx and RLx:** Each local controller uses a sensor-input thread for blocking keyboard input and a watchdog thread that checks whether the server's periodic FSM processing continues to advance.

Periodic events

The controllers use POSIX timers configured with `CLOCK_REALTIME`. Timer expiry delivers a `SIGEV_PULSE` to the controller's IPC channel, allowing the main server loop to process timer events and IPC requests sequentially.

- **C1 - 1 s:** Checks controller availability, evaluates scheduled mode selection, and refreshes the network-status display.
- **Lx - 100 ms:** Advances vehicle phases, pedestrian indications, railway preemption, overrides, and related local timing.
- **Lx - 1 s:** Initiates a heartbeat to C1 and checks local-clock mode selection when the Central link is degraded.
- **RLx - 1 s crossing-FSM timer:** Advances warning, gate movement, expected-arrival, occupancy, and opening countdowns.

- **RLx - 1 s heartbeat timer:** Initiates a separate heartbeat to C1.

Table 8. Concurrency and synchronization summary

Shared resource or event	Producer(s)	Consumer(s)	Synchronization mechanism	Rationale and limitation
Incoming IPC requests and timer pulses	Other controllers and POSIX timers	IPC server thread	One <code>MsgReceive()</code> loop dispatches requests and pulses; only the server thread calls <code>MsgReply()</code> .	Request and pulse handlers execute sequentially within each controller. FSM mutexes remain necessary because sensor, client-callback, and watchdog threads can access the same state concurrently.
Outgoing IPC queue	C1 operator and server logic; Lx and RLx server logic	Dedicated IPC client thread	A 16-entry FIFO protected by a mutex and condition variable. The client removes one job and performs <code>name_open()</code> and <code>MsgSend()</code> after releasing the queue lock.	Prevents blocking outgoing IPC from stopping the server loop. If the queue is full, a new job is rejected; successful delivery is not guaranteed.
C1 network view and schedule state	Server thread and operator thread	Server thread and operator thread	<code>mode_eng_lock</code> protects the shared mode-engine state.	Prevents concurrent operator, incoming-message, and periodic-timer access from producing inconsistent controller records or schedule state.
C1 terminal and log output	Server thread, operator thread, and client reply callbacks	Terminal and <code>central_log.txt</code>	<code>console_io_lock</code> serializes prompts, status tables, and log output. When both C1 locks are required, <code>console_io_lock</code> is acquired before <code>mode_eng_lock</code> .	Prevents terminal output from interleaving and maintains a consistent lock-acquisition order. The operator releases the console lock while waiting for keyboard input.
Lx FSM state	Server, sensor, client-callback, and watchdog threads	The same threads	The mutex in <code>lx_fsm_t</code> protects phase, pedestrian, override, fault, sensor, and link-state fields.	Keeps state transitions and heartbeat snapshots consistent while local input, IPC, timers, and watchdog checks occur concurrently.
RLx FSM and simulated gate state	Server, sensor, client-callback, and watchdog threads	The same threads	The mutex in <code>rlx_fsm_t</code> protects crossing state. A separate gate mutex protects gate motion and confirmation flags. When both are required, the gate lock is acquired while the FSM lock is already held.	Prevents conflicting crossing and gate updates. No code path acquires the FSM lock while already holding the gate lock, avoiding a reversed two-lock order.

4. Inter-Process Communication

4.1 Communication Topology

4.1.1 Name Attachment and Discovery

Each controller calls `ipc_attach()` during startup to register one named attach point using the format:

```
traffic/<suffix>
```

The suffix identifies the controller, for example `traffic/c1`, `traffic/l1`, or `traffic/r11`. A complete ten-controller deployment therefore creates **ten attach points**, one per running controller process.

`ipc_attach()` first attempts to register the name using `NAME_FLAG_ATTACH_GLOBAL`. If the global name service is unavailable, it falls back to the local namespace. This allows the same controller code to operate with either global or local QNX name attachment.

4.1.2 Dynamic Node Resolution (TRAFFIC_NODE_MAP)

Cross-node destinations are resolved at runtime through the `TRAFFIC_NODE_MAP` environment variable. The variable contains comma-separated mappings from controller suffixes to Qnet node names, for example:

```
c1=<node>, l1=<node>, ..., rl3=<node>
```

The mapping is parsed on the first destination lookup and cached for the lifetime of the process.

- **Mapped remote target:** The client first opens `/net/<node>/dev/name/global/traffic/<suffix>`. If the global path cannot be opened, it attempts the corresponding `/dev/name/local/` path on that Qnet node.
- **Unmapped target:** If the destination suffix is absent from `TRAFFIC_NODE_MAP`, or the variable is unset, the client uses the same-node name `traffic/<suffix>`.

This resolution allows the controller binaries to be deployed on different Qnet nodes without recompilation. A changed mapping takes effect only after the process is restarted.

4.1.3 Communication Roles and Authority

The system implements three inter-controller communication relationships:

- **Central to intersection - C1 ↔ Lx:** C1 sends operating-mode, timing-profile, override, renewal, cancellation, and fault-clear requests. Each Lx validates the request against its local state and returns a synchronous reply. Lx sends a periodic heartbeat containing its current status to C1. C1 cannot directly set road or pedestrian signal outputs.
- **Central to railway - C1 ↔ RLx:** C1 can send a fault-clear request to an RLx. RLx sends heartbeats, crossing-status updates, and fault reports to C1. C1 cannot directly command gates, warning flashers, or train signals.
- **Railway to intersection - RLx → Lx:** Each RLx sends crossing-status updates directly to its two adjacent Lx controllers. This is a **unidirectional application-level status feed with a synchronous transport ACK**. Lx uses the reported crossing state for local railway preemption but has no command authority over RLx.

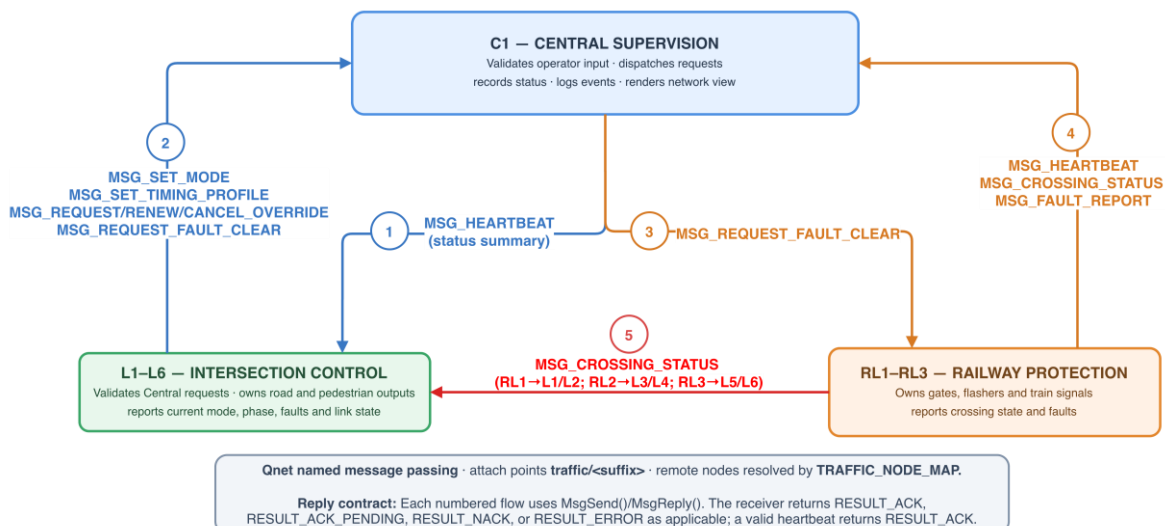


Figure 4. Implemented Qnet IPC message flows

4.1.4 Dedicated IPC Client Thread

QNX `MsgSend()` blocks until the destination returns `MsgReply()`. Because every controller must remain available to receive incoming requests and pulses, outgoing sends are handled by a dedicated IPC client thread.

Application and FSM code place outgoing requests into a **16-entry FIFO queue** using `ipc_client_post()`. The client thread removes one request at a time and performs `name_open()`, `MsgSend()`, and `name_close()` outside the queue lock.

This prevents an outgoing synchronous send from blocking the controller's receive loop. If the queue is full, `ipc_client_post()` rejects the new job and returns immediately. Message delivery is therefore not guaranteed merely because the application attempted to enqueue a request.

4.1.5 Asynchronous Pulse Timers

`ipc_timer_arm()` creates POSIX timers that deliver `SIGEV_PULSE` events to the receiving controller's IPC channel. The implemented pulse types are:

- **IPC_PULSE_PHASE_TIMER - 100 ms, Lx:** Advances the intersection FSM, including vehicle phases, pedestrian indications, override timing, railway preemption, and demand-based extensions.
- **IPC_PULSE_HEARTBEAT_TICK - 1 s, all controller types:** Initiates Lx and RLx heartbeats. At C1, the pulse drives availability checks, scheduled-mode evaluation, and network-status display refresh. At Lx, it also triggers local-clock mode selection when the Central link is degraded.
- **IPC_PULSE_RAILWAY_WARNING - 1 s, RLx:** Advances the railway crossing FSM, including the 5 s warning-to-closing interval, gate-confirmation deadlines, expected-arrival delay, occupancy windows, and opening sequence.

Timer pulses arrive from `MsgReceive()` with `rcvid == 0`. They are dispatched locally and do not receive a `MsgReply()`.

4.1.6 Heartbeat Cadence and State Resynchronization

Each Lx and RLx uses a **1 s heartbeat timer** to trigger `MSG_HEARTBEAT` delivery to C1. The heartbeat carries the controller's current status summary.

C1 records a valid heartbeat summary and returns `RESULT_ACK`. The result updates the sender's Central-link state as follows:

- **Connection:** A valid heartbeat ACK changes the sender's link state to `LINK_CENTRAL_CONNECTED` and resets its recorded missed-ACK count.
- **Degradation:** After three completed heartbeat attempts are recorded without a valid ACK, a previously connected controller changes to `LINK_DEGRADED_LOCAL`.
- **Local operation:** Lx and RLx continue their local FSMs while degraded. Lx additionally uses its local clock for scheduled Peak or Off-Peak mode selection.
- **Status refresh after reconnection:** The first successfully delivered heartbeat updates C1 with the sender's current status summary. The returned ACK restores the sender's link state to `LINK_CENTRAL_CONNECTED`.

This mechanism provides a **heartbeat-carried status refresh** without requiring a separate state-synchronization message. It does not replay an RLx crossing-state update that an adjacent Lx may have missed while disconnected.

Because synchronous `MsgSend()` has no configured timeout, a destination process that remains present but stops processing messages can block the sender's IPC client thread and delay link-degradation detection. This limitation is recorded in **Section 8.4**.

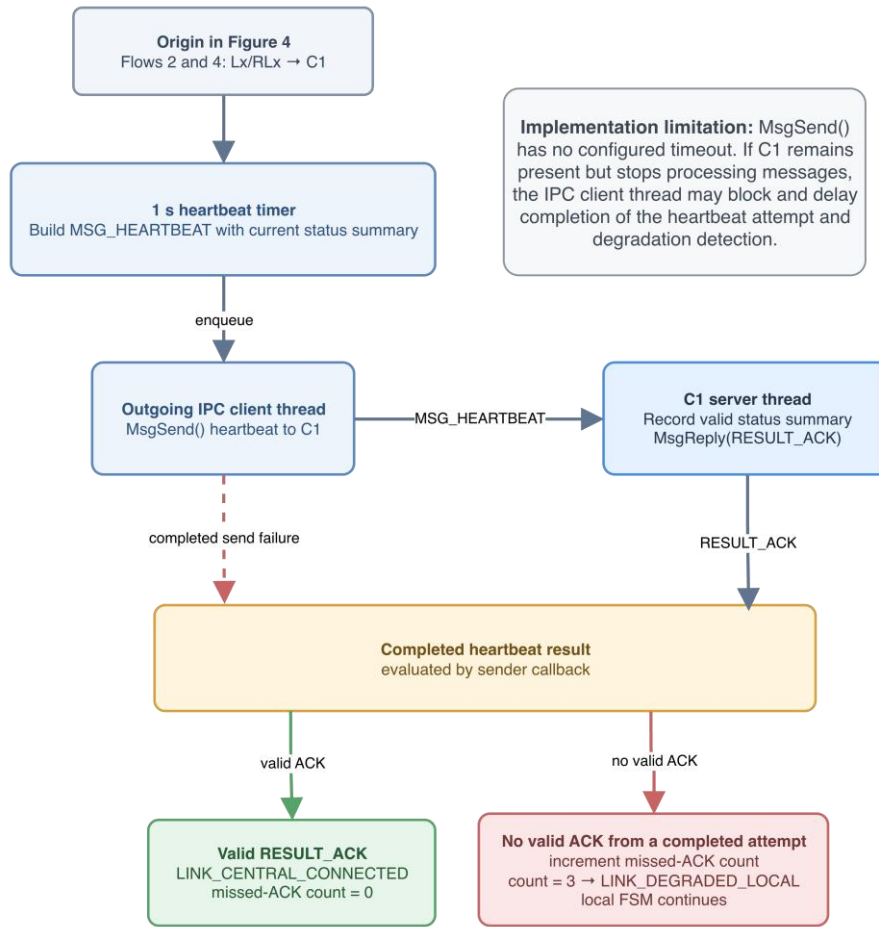


Figure 5. Heartbeat delivery and Central-link state update

4.2 Message Pathways

Table 9 lists the implemented application messages and asynchronous pulses. For each pathway, it identifies the sender, receiver, transport mechanism, operational purpose, and reply to behavior.

Table 9. Implemented inter-process communication pathways

Message / pulse	Source	Destination	Mechanism	Operational purpose	Reply / timing behavior
MSG_SET_MODE	C1 (<code>c_comm.c</code>)	Selected Lx or all Lx	<code>MsgSend()</code> / <code>MsgReply()</code> through the outgoing IPC queue	Requests <code>PEAK_FIXED</code> or <code>OFF_PEAK_SENSOR</code> . A different mode is stored as pending and applied at a safe phase boundary.	<code>RESULT_ACK</code> if the requested mode is already active; <code>RESULT_ACK_PENDING</code> for a deferred change; otherwise, <code>RESULT_NACK(reason)</code> .
MSG_SET_TIMING_PROFILE	C1 (<code>c_comm.c</code>)	Selected arterial Lx controllers	<code>MsgSend()</code> / <code>MsgReply()</code> through the outgoing IPC queue	Supplies the profile ID and assigned green-wave offset. The accepted offset is applied at the next appropriate arterial- green boundary.	<code>RESULT_ACK</code> if accepted; <code>RESULT_NACK(reason)</code> for a fault-active controller or an invalid offset.
MSG_REQUEST_OVERRIDE	C1 (<code>c_comm.c</code>)	Selected Lx	<code>MsgSend()</code> / <code>MsgReply()</code> through the outgoing IPC queue	Requests a bounded clear-route override for the arterial or connector movement. Lx validates the duration, movement, current supervisory state, railway state, and	<code>RESULT_ACK</code> if activated immediately; <code>RESULT_ACK_PENDING</code> while pedestrian clearance completes; otherwise, <code>RESULT_NACK(reason)</code> .

				pedestrian clearance.	
MSG_RENEW_OVERRIDE	C1 (c_comm.c)	Selected Lx	MsgSend() / MsgReply() through the outgoing IPC queue	Restarts the countdown of an active override. A non-zero value becomes the new duration; 0 reuses the stored duration. It does not add time to the current remaining value.	RESULT_ACK if an active override is renewed; otherwise RESULT_NACK(reason).
MSG_CANCEL_OVERRIDE	C1 (c_comm.c)	Selected Lx	MsgSend() / MsgReply() with no application payload	Cancels an active or pending override. Lx enters the required clearance sequence before normal operation resumes.	RESULT_ACK if an override was active or pending; otherwise RESULT_NACK(NACK_REASON_UNKNOWN_TARGET).
MSG_REQUEST_FAULT_CLEAR	C1 (c_comm.c)	Selected Lx or RLx	MsgSend() / MsgReply() with no application payload	Lx: clears its fault flags and ACKs. If it was in FAULT_SAFE, it resumes railway preemption when the last crossing state is not OPEN; otherwise it resumes normal operation. RLx: clears the fault only after confirming that it is in FAULT and the simulated gate is currently confirmed OPEN.	Lx: unconditional RESULT_ACK. RLx: RESULT_ACK after successful confirmed-open validation; otherwise RESULT_NACK(NACK_REASON_FAULT_ACTIVE) or RESULT_NACK(NACK_REASON_UNKNOWN_TARGET).
MSG_STATUS	No implemented runtime sender	C1 (c_main.c)	Defined in ipc_msg.h and accepted by C1 through MsgSend() / MsgReply()	Provides a standalone status-report format retained in the protocol. The implemented Lx and RLx processes do not send this message; their status summaries are carried in MSG_HEARTBEAT.	C1 returns RESULT_ACK if such a valid message is received.
MSG_FAULT_REPORT	RLx (rlx_comm.c)	C1	MsgSend() / MsgReply() through the outgoing IPC queue	Reports a railway fault to C1 while RLx independently applies its local fail-safe behavior. C1 records the report and appends a log entry.	C1 returns RESULT_ACK.
MSG_CROSSING_STATUS	RLx (rlx_comm.c)	Two adjacent Lx controllers and C1	MsgSend() / MsgReply() through the outgoing IPC queue	Provides a unidirectional application-level status feed containing OPEN, WARNING, CLOSED, or FAULT. Adjacent Lx controllers use it to suppress conflicting movements; C1 uses it for monitoring.	Each receiver returns a synchronous transport RESULT_ACK. Lx does not return a railway command.
MSG_HEARTBEAT	Lx (lx_comm.c) and RLx (rlx_comm.c)	C1	Triggered by a 1 s pulse and delivered through the outgoing IPC queue using MsgSend() / MsgReply()	Carries the sender's current status summary for Central monitoring and link-state maintenance. The first heartbeat delivered after reconnection refreshes C1's view.	A valid heartbeat expects RESULT_ACK. A valid ACK sets LINK_CENTRAL_CONNECTED and resets the missed-ACK count. Three completed heartbeat attempts without a valid ACK change a previously connected sender to LINK_DEGRADED_LOCAL. A blocking MsgSend() can delay completion and therefore delay degradation detection.
IPC_PULSE_PHASE	Lx POSIX	Same Lx	SIGEV_PULSE	Advances the local signal-	Kernel pulse; no reply.

SE_TIMER	timer	process	every 100 ms	phase FSM, pedestrian timing, sensor-driven demand handling, override countdown, and watchdog progress counter.	
IPC_PULSE_HEARTBEAT_TICK	POSIX timer on every controller	Same controller process	SIGEV_PULSE every 1 s	On Lx and RLx, triggers heartbeat construction and queueing. On C1, triggers controller-liveness scanning, scheduled-mode evaluation, and HMI refresh.	Kernel pulse; no reply.
IPC_PULSE_RAILWAY_WARNING	RLx POSIX timer	Same RLx process	SIGEV_PULSE every 1 s	Acts as the implemented RLx crossing-FSM tick. It advances the warning-to-closing interval, gate-confirmation deadlines, expected-arrival delay, occupancy windows, opening sequence, fault-report checks, and crossing-status checks.	Kernel pulse; no reply.

4.3 Message Validation and Failure Handling

4.3.1 Wire-Format Contract and Alignment

All IPC payloads are wrapped in `ipc_request_t` and `ipc_reply_t` structures. The message header matches the binary layout of QNX Neutrino's `struct _pulse`, preventing memory corruption from pointer size variations (such as 32-bit vs. 64-bit `sival_ptr`) when pulses enter the server buffer.

Fixed-Width Types: All fields crossing node boundaries use explicit fixed-width integers (`uint8_t`, `uint32_t`, `uint64_t`). C enums are used for code clarity, but wire data is strictly serialized as explicit integers to avoid compiler-dependent enum sizing issues across platforms.

4.3.2 Demultiplexing and Verb-Based Dispatch

Each controller listens on a single channel. Incoming requests are demultiplexed using the `verb` field in `ipc_request_t`, mapping directly to the appropriate FSM handler in `on_request()`. This is kept separate from `hdr.type`, which handles system I/O (`_IO*`). Client routines explicitly zero `hdr.type` before sending to prevent collisions with system messages.

4.3.3 Autonomous FSM Safety Checks

Field controllers follow defensive programming principles (PA-09) and never execute commands blindly:

- **Operating Mode Bounds:** `lx_fsm_on_set_mode()` verifies that requested modes strictly equal `MODE_PEAK_FIXED` or `MODE_OFF_PEAK_SENSOR`. Invalid values are rejected with `RESULT_NACK(NACK_REASON_OUT_OF_RANGE)`.
- **Override Validation:** `lx_fsm_on_request_override()` checks that the target movement is valid, bounds duration between 1 and 300 seconds (PA-11), and blocks nested override attempts.
- **Hardware Fault Re-Verification:** Central requests to clear faults (`rlx_fsm_on_fault_clear()`) are rejected if the crossing is not in a fault state. If faulted, the controller re-checks live gate sensors via `gates_confirmed_open()`. Clearance is rejected with `NACK_REASON_FAULT_ACTIVE` if hardware is not proven open (RC-09).

4.3.4 Negative Acknowledgement (NACK) Taxonomy

A controller returns `RESULT_NACK` when a request is understood but cannot be applied safely or validly. The reply's `reason` field contains a `nack_reason_t` value.

Reasons returned by the implemented handlers:

- **NACK_REASON_INVALID_DURATION:** An override duration is zero, exceeds the 300,000 ms cap, or a renewal duration exceeds that cap.
- **NACK_REASON_RAILWAY_CONFLICT:** An override conflict with active railway preemption.
- **NACK_REASON_FAULT_ACTIVE:** The requested operation cannot be applied while the target controller is faulted. RLx also returns this reason when fault clearance is requested without confirmed-open gate feedback.
- **NACK_REASON_STALE_OR_UNSAFE_PROFILE:** A timing-profile offset is greater than or equal to the 90,000 ms cycle length.
- **NACK_REASON_OUT_OF_RANGE:** A mode or movement value is outside its supported range. Lx also uses this reason when a new override is requested while another override is already active or pending.
- **NACK_REASON_UNKNOWN_TARGET:** The requested state-dependent operation has no applicable target, such as renewing or cancelling an inactive override or clearing an RLx that is not in `FAULT`.

Defined but not currently returned:

- **NACK_REASON_PEDESTRIAN_ACTIVE:** This reason remains defined in `sys_types.h`, but the implemented override handler does not return it. When pedestrian clearance is active, a valid override is retained as pending and Lx returns `RESULT_ACK_PENDING`.

`RESULT_ERROR` is separate from the NACK taxonomy. It is used when a received message verb is unsupported or cannot be processed as a valid request.

4.3.5 One-Second Response Deadline (PA-12)

The implementation is structured to support the **one-second supervisory response target**:

- **Bounded server handling:** Request handlers perform mutex-protected, in-memory validation and state updates. They do not perform outgoing IPC or wait for a later FSM transition before replying.
- **Immediate acceptance result:** A request returns `RESULT_ACK`, `RESULT_ACK_PENDING`, `RESULT_NACK`, or `RESULT_ERROR` after validation.
- **Deferred activation:** `RESULT_ACK_PENDING` confirms that a valid request was accepted while its effect is deferred until a safe transition, such as completion of pedestrian clearance or arrival at a safe mode-change boundary.
- **Dedicated outgoing IPC thread:** Blocking `name_open()` and `MsgSend()` calls execute on the client thread and therefore do not block the receiver's IPC server loop.

The implementation does not configure a transport timeout or enforce a measured worst-case response bound. In particular, `MsgSend()` can remain blocked when a destination process still exists but does not process messages. **Section 8.3** therefore reports the one-second requirement as an **observed response target**, not a guaranteed deadline.

4.3.6 Autonomous Railway Protection Under Faults (RC-10)

Railway fault handling is performed locally by RLx and does not require a command or acknowledgement from C1.

When a railway fault is detected, `enter_fault()` synchronously performs the following local updates:

- Commands the simulated gates to close through `rlx_gate_command_close()`.
- Calls `rlx_signal_show_fault()` to indicate that the fault is latched and all train signals are held at STOP.
- Changes the local crossing FSM state to `RLX_FAULT`.
- Resets the state elapsed-time counter.
- Latches the applicable fault flag.
- Sets `fault_report_pending` for asynchronous reporting to C1.

The recurring RLx FSM tick subsequently consumes the pending-report flag and queues `MSG_FAULT_REPORT` through the outgoing IPC client. A failure to reach C1 therefore does not prevent the local crossing from entering or remaining in its fault state.

No execution-time bound was measured for `enter_fault()` or the simulated output calls. The implementation therefore demonstrates **synchronous local fail-safe handling**, without claiming microsecond response performance.

5. Source-Code Organization

This section details the organization of the submitted source code. It defines the specific directory, file, and module-level responsibilities that drive the central, intersection, and railway controllers.

5.1 Submitted QNX Workspace Structure

The *Traffic_Control_System* working set contains three distinct QNX Momentics projects. Each project can be imported and built independently and contains the source required by its controller type. **Figure 6** details the standard directory structure utilized across the repository.

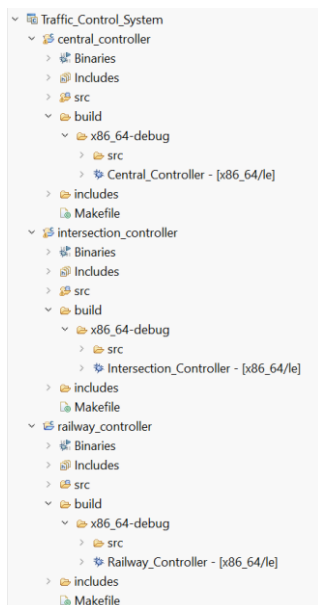


Figure 6. QNX Momentics workspace organization and standard directory responsibility

Directory & File Responsibilities:

- **Generated build output:** The **Momentics Binaries** view and `build/x86_64-debug/` directory expose generated executables, object files, and other build artifacts. These files are produced by the QNX toolchain and are not part of the team-authored source listing below.
- **src/** Contains all source code implementations (c files) specific to that node's domain logic, state machines, and IPC handling.
- **includes/** Contains the project header files (.h). This includes controller-specific definitions as well as the shared IPC message contracts (`ipc_msg.h`, `sys_types.h`) utilized for cross-node Qnet communication.
- **Makefile** The standard QNX build instructions used by the Momentics IDE to compile and link the project binaries.
- Each submitted project contains its own copy of `includes/ipc_msg.h`, `includes/sys_types.h`, `includes/qnet_utils.h`, and `src/qnet_utils.c`. The projects do not link against a separate shared library.

5.2 Project, File, and Module Responsibilities

The submitted workspace contains **three independently buildable QNX Momentics projects**. Each project contains its controller-specific modules and its own copy of the common IPC contracts, system types, and Qnet utility source. The projects do not link against a separate shared library.

- **Central_Controller/**: Provides network supervision, operator commands, status display, event logging, scheduled-mode selection, and command distribution. It does not directly actuate road or railway equipment.
- **Intersection_Controller/**: Provides autonomous vehicle and pedestrian sequencing, sensor-driven operation, supervisory overrides, railway preemption, and local fault handling.
- **Railway_Controller/**: Provides autonomous crossing protection, simulated gate and signal control, train-occupancy tracking, fault handling, and direct crossing-status delivery.

The tables below list the team authored `.c` and `.h` modules. Generated object files and the Momentics Binaries and Includes views are omitted. Each project's submitted Makefile provides its build configuration, as described in Section 2.

Table 10. Shared IPC Contracts & Utilities (Replicated across all projects)

Path	Purpose	Used by
<code>sys_types.h</code>	Defines controller IDs and roles, operating modes, signal and crossing states, supervisory and link states, fault flags, NACK reasons, and other system-wide types.	Controller-specific headers and source modules in all three projects.
<code>ipc_msg.h</code>	Defines message verbs, request and reply envelopes, status summaries, and payload structures exchanged between C1, Lx, and RLx.	IPC handlers, communication modules, FSM interfaces, and Qnet utilities in all three projects.
<code>qnet_utils.h</code>	Declares attach-point creation, IPC server callbacks, the outgoing client queue, reply callbacks, Qnet node resolution, and pulse-timer helpers.	Controller entry points and communication modules in all three projects.
<code>qnet_utils.c</code>	Implements Qnet attach and lookup operations, <code>TRAFFIC_NODE_MAP</code> resolution, the IPC server loop, the 16-entry outgoing FIFO, the dedicated blocking <code>MsgSend()</code> client thread, reply delivery, and POSIX pulse-timer creation.	Compiled separately into each of the three QNX projects.

Table 11. Central Controller Project (*central_controller/*)

Path	Purpose	Used by
<code>c_main.c</code>	Program entry point. Creates the Central IPC server and client infrastructure, operator thread, mode engine, monitoring state, HMI, and logging context. The Momentics project build determines the executable filename.	Project entry point; not included by another module.
<code>c_operator.h</code> <code>c_operator.c</code>	Runs the keyboard operator thread and parses mode, timing-profile, override, fault-clear, and simulated-hour commands.	Created by <code>c_main.c</code> ; calls the outgoing functions in <code>c_comm.c</code> .
<code>c_comm.h</code> <code>c_comm.c</code>	Builds and queues mode, timing-profile, override, renewal, cancellation, and fault-clear requests. Processes their completed replies for operator-visible output and logging.	Used by <code>c_operator.c</code> , <code>c_main.c</code> , and scheduled-mode processing.
<code>c_mode_eng.h</code> <code>c_mode_eng.c</code>	Maintains C1's current controller-status view, availability and scheduling state, simulated-hour selection, and arterial timing-profile construction.	Used by <code>c_main.c</code> , <code>c_operator.c</code> , <code>c_comm.c</code> , <code>c_server.c</code> , <code>c_watchdog_mon.c</code> , and <code>c_hmi.c</code> .
<code>c_server.h</code> <code>c_server.c</code>	Updates C1's current controller and crossing-status view from heartbeat summaries, standalone status messages, and railway crossing reports.	Called by the inbound request handler in <code>c_main.c</code> . Fault-report logging remains in <code>c_main.c</code> .
<code>c_watchdog_mon.h</code> <code>c_watchdog_mon.c</code>	Performs the 1 Hz availability scan and marks a local controller <code>UNAVAILABLE</code> after three consecutive missed checks.	Called by the 1 Hz pulse handler in <code>c_main.c</code> .
<code>c_hmi.h</code> / <code>c_hmi.c</code>	Renders the current network-status table to the Central terminal.	Called by <code>c_main.c</code> during the 1 Hz Central update while holding the console lock.
<code>c_logger.h</code> / <code>c_logger.c</code>	Writes timestamped system events to standard output and, when available, <code>central_log.txt</code> .	Used by <code>c_main.c</code> , <code>c_operator.c</code> , and <code>c_comm.c</code> .

Table 12. Intersection Controller Project (*intersection_controller/*)

Path	Purpose	Used by
<code>lx_main.c</code>	Program entry point. Parses the L1–L6 instance ID and creates the local IPC, timer, sensor-input, and watchdog execution context. The Momentics project build determines the executable filename.	Project entry point; not included by another module.
<code>lx_fsm.h / lx_fsm.c</code>	Implements vehicle-phase sequencing, pedestrian service, sensor-driven operation, supervisory mode changes, overrides, railway preemption, link-state behavior, and local fault handling.	Called by <code>lx_main.c</code> , <code>lx_sensor.c</code> , <code>lx_comm.c</code> , and <code>lx_watchdog.c</code> .
<code>lx_timer.h / lx_timer.c</code>	Provides configured phase-duration lookups and the off-peak minimum-green, demand-extension, and exit calculations used by the FSM.	Used by <code>lx_fsm.c</code> .
<code>lx_comm.h / lx_comm.c</code>	Builds and queues the 1 Hz heartbeat containing the current Lx status summary. Updates the Central-link state from the completed heartbeat result.	Called by the 1 Hz pulse handler in <code>lx_main.c</code> .
<code>lx_sensor.h / lx_sensor.c</code>	Reads simulated arterial and connector vehicle presence, pedestrian-button presses, and advance queue-warning inputs from the terminal.	Sensor thread created by <code>lx_main.c</code> ; updates <code>lx_fsm.c</code> .
<code>lx_signal.h / lx_signal.c</code>	Displays simulated road-signal and pedestrian-indication outputs in the Lx terminal.	Called by <code>lx_fsm.c</code> during state transitions.
<code>lx_watchdog.h / lx_watchdog.c</code>	Monitors phase-timer progress and invokes the local fail-safe handler if the control loop stops advancing.	Watchdog thread created by <code>lx_main.c</code> ; updates <code>lx_fsm.c</code> .

Table 13. Railway Controller Project (*railway_controller/*)

Path	Purpose	Used by
<code>rlx_main.c</code>	Program entry point. Parses the RL1–RL3 instance ID and creates the local IPC, timer, sensor-input, and watchdog execution context. The Momentics project build determines the executable filename.	Project entry point; not included by another module.
<code>rlx_fsm.h / .c</code>	Implements the crossing-protection sequence, expected-arrival timing, overlapping train-occupancy windows, gate confirmation, opening and reclosing, fault handling, and recovery.	Called by <code>rlx_main.c</code> , <code>rlx_sensor.c</code> , <code>rlx_comm.c</code> , and <code>rlx_watchdog.c</code> .
<code>rlx_gate.h / .c</code>	Simulates aggregate gate motion and open/closed confirmation used by the confirmation-before-PROCEED invariant.	Called by <code>rlx_fsm.c</code> ; demonstration fault and repair state is controlled through <code>rlx_sensor.c</code> .
<code>rlx_signal.h / .c</code>	Displays simulated warning-flasher, train-signal, reclosing, and fault indications in the RLx terminal.	Called by <code>rlx_fsm.c</code> during crossing transitions and faults.
<code>rlx_sensor.h / .c</code>	Reads simulated train approaches and demonstration controls for gate-confirmation failure, confirmed-open repair feedback, and the demo-only local fault-clear path.	Sensor thread created by <code>rlx_main.c</code> ; updates <code>rlx_fsm.c</code> and the simulated gate state.
<code>rlx_comm.h / .c</code>	Sends changed crossing status to both adjacent Lx controllers and C1, queues 1 Hz heartbeat summaries, and sends railway fault reports to C1.	Called by the timer and fault-report handling paths in <code>rlx_main.c</code> .
<code>rlx_timer.h / .c</code>	Provides countdown-window helpers used for expected-arrival and overlapping train-occupancy timing.	Used by <code>rlx_fsm.c</code> .
<code>rlx_watchdog.h / .c</code>	Monitors RLx FSM progress and invokes the local safe-fault path if the control loop stops advancing.	Watchdog thread created by <code>rlx_main.c</code> ; updates <code>rlx_fsm.c</code> .

6. Key Implementation Decisions

This section records the principal engineering decisions made during implementation and their practical consequences. **Sections 3–5** describe the implemented architecture and modules; **Table 14** explains why the following implementation choices were made.

Table 14. Key Implementation Decisions

Decision	Reason	Consequence / trade-off
Local output authority	C1 sends supervisory requests but does not directly actuate road signals, pedestrian indications, gates, flashers, or train signals. Each local controller validates incoming requests before changing its own state or outputs.	A C1 or communication failure cannot directly take ownership of local outputs. A rejected request must be corrected and submitted again by C1 or the operator.
Direct	Crossing state is sent directly from each RLx to its	Adjacent intersections can suppress conflicting road

railway preemption path	two adjacent Lx controllers so that intersection preemption does not depend on C1 receiving and relaying the event.	movements independently of C1. A crossing-state update missed by a disconnected Lx is not replayed until RLx changes state again.
Synchronous command replies	Each receiving controller performs bounded local validation and returns <code>RESULT_ACK</code> , <code>RESULT_ACK_PENDING</code> , <code>RESULT_NACK</code> , or <code>RESULT_ERROR</code> . <code>RESULT_ACK_PENDING</code> acknowledges a valid request whose activation must wait for a safe state boundary.	The operator receives the request outcome separately from its eventual activation. The structure supports the one-second response target, but no transport timeout or worst-case response bound is enforced.
Console I/O lock management	<code>console_io_lock</code> serializes Central prompts, reply messages, logs, and HMI output. It is always acquired outside <code>mode_eng_lock</code> and is released while <code>scanf()</code> waits for keyboard input.	Blocking keyboard input does not prevent the 1 Hz server-side status refresh. Because the lock is released while waiting, an HMI table or event message can appear between an operator prompt and the entered value.
Heartbeat-carried status refresh	The existing heartbeat includes the controller's current status summary, avoiding a separate reconnect handshake or state-synchronization message.	The next successfully delivered heartbeat restores C1's status view and the returned ACK restores the sender's connected link state. This does not replay a crossing-state change that an adjacent Lx missed while disconnected.
Live peak-hour automatic switching	C1 evaluates the current or forced demonstration hour on its 1 Hz tick and broadcasts a mode request when the required Peak or Off-Peak mode changes.	The broadcast consists of six independent unicast requests. Each Lx validates the request and can apply a changed mode at a different safe phase boundary.
Minimum-green preservation during railway preemption	Once railway preemption is active, an existing green connector is allowed to satisfy its configured minimum-green constraint before transitioning through the required clearance sequence. New connector green phases are suppressed until the crossing reopens.	The implementation avoids an abrupt unsafe-looking green truncation while still preventing another conflicting connector phase from starting during the railway closure. Preemption completion therefore follows the local minimum-green and clearance timing rather than an immediate output change.
Dynamic Qnet node resolution	<code>TRAFFIC_NODE_MAP</code> maps controller suffixes to Qnet node names at runtime, allowing the same binaries to communicate across the ten QNX VMs without recompilation for a different controller-to-VM mapping.	The map is parsed once and cached for the process lifetime. Changing a controller's Qnet location requires restarting the affected process. An unmapped destination falls back to same-node lookup.

Implementation Locations:

- Local output authority:** `Intersection_Controller/src/lx_fsm.c` and `Intersection_Controller/src/lx_signal.c` implement and display local road and pedestrian outputs. `Railway_Controller/src/rlx_fsm.c`, `Railway_Controller/src/rlx_gate.c`, and `Railway_Controller/src/rlx_signal.c` implement and display local crossing outputs.
- Direct railway preemption path:** `Railway_Controller/src/rlx_comm.c` (`rlx_comm_broadcast_crossing_status_if_changed()` and `send_crossing_status()`) sends crossing state directly to adjacent intersections. `Intersection_Controller/src/lx_fsm.c` (`lx_fsm_on_crossing_status()`) applies the received state to local preemption.
- Synchronous command replies:** `Intersection_Controller/src/lx_fsm.c` and `Railway_Controller/src/rlx_fsm.c` validate requests and select the reply result. The replicated `src/qnet_utils.c` receives requests, dispatches their handlers, and completes the synchronous reply with `MsgReply()`.
- Console I/O lock management:** `Central_Controller/src/c_operator.c` (`read_long()` and `c_operator_reader_thread()`) and `Central_Controller/src/c_main.c` (`on_pulse()`).
- Heartbeat-carried status refresh:** `Intersection_Controller/src/lx_comm.c` and `Railway_Controller/src/rlx_comm.c` build and queue heartbeat summaries. `Intersection_Controller/src/lx_fsm.c` (`lx_fsm_on_heartbeat_result()`) and `Railway_Controller/src/rlx_fsm.c` (`rlx_fsm_on_heartbeat_result()`) update the sender's Central-link state. The heartbeat payload is defined in the replicated `includes/ipc_msg.h`.

- **Live peak-hour automatic switching:** `Central_Controller/src/c_main.c` (`on_pulse()`) and `Central_Controller/src/c_mode_eng.c` (`c_mode_eng_auto_check()`).
- **Minimum-green preservation during railway preemption:** `Intersection_Controller/src/lx_fsm.c` (`lx_fsm_on_crossing_status()` and `lx_fsm_on_phase_timer()`).
- **Dynamic Qnet node resolution:** the replicated `src/qnet_utils.c` implements `TRAFFIC_NODE_MAP` parsing, cached controller-to-node mapping, and destination path construction for all three projects.

7. Implemented Features and Operational Evidence

This section provides operational verification and terminal evidence for the core and high-distinction capabilities of the distributed traffic control system. Representative terminal logs captured across the QNX Neutrino nodes demonstrate that the implementation satisfies all real-time requirements, safety invariants, and operational scenarios defined in the course specification. Each subsection details the input trigger sequence, internal state machine transitions, timing parameters, and observable safety outcomes.

7.1 Normal traffic and pedestrian operation

7.1.1 Deterministic Peak Fixed-Timing Baseline (UC-01, TL-01)

During high-density peak traffic hours, intersection controllers operate in `MODE_PEAK_FIXED` to maintain deterministic, coordinated network flow. As defined in `lx_timer.h`, the controller enforces a 90-second cycle length (`LX_CYCLE_LENGTH_MS = 90000 ms`) split into major arterial and minor connector stages:

- **Arterial Stage:** 48.0 s green (`LX_PEAK_ARTERIAL_GREEN_MS`), followed by a 4.0 s amber clearance (`LX_YELLOW_MS`) and a 2.0 s all-red interlock interval (`LX_ALL_RED_MS`).
- **Connector Stage:** 30.0 s green (`LX_PEAK_CONNECTOR_GREEN_MS`), followed by a 4.0 s amber clearance and a 2.0 s all-red interlock interval.
- Phase transitions are driven by asynchronous kernel pulses (`IPC_PULSE_PHASE_TIMER`) every 100 ms. Operational evidence confirms that inter-phase clearance intervals are strictly maintained without timing drift, guaranteeing that conflicting movements never display green simultaneously (TL-01).

Figure 7 displays the deterministic signal phase progression of local intersection controller L1 operating in baseline peak mode (`MODE_PEAK_FIXED`), validating requirements TL-01 and UC-01. Paced by periodic 100 ms asynchronous kernel pulses (`IPC_PULSE_PHASE_TIMER`), the controller executes a continuous 90.0-second cycle split into arterial service (48.0 s green) and connector service (30.0 s green). As evidenced across the multiple consecutive cycles in the terminal trace, transitions between opposing movements strictly enforce intermediate clearance intervals: a 4.0 s amber clearance (`LX_YELLOW_MS`) followed by a mandatory 2.0 s all-red interlock (`LX_ALL_RED_MS`). The presence of `ALL RED (A to B)` and `ALL RED (B to A)` prior to every green activation proves that conflicting vehicle paths are physically isolated, preventing simultaneous green allocations under all operational conditions. The continuous repetition confirms that kernel pulse dispatch maintains deterministic timing without cumulative drift.

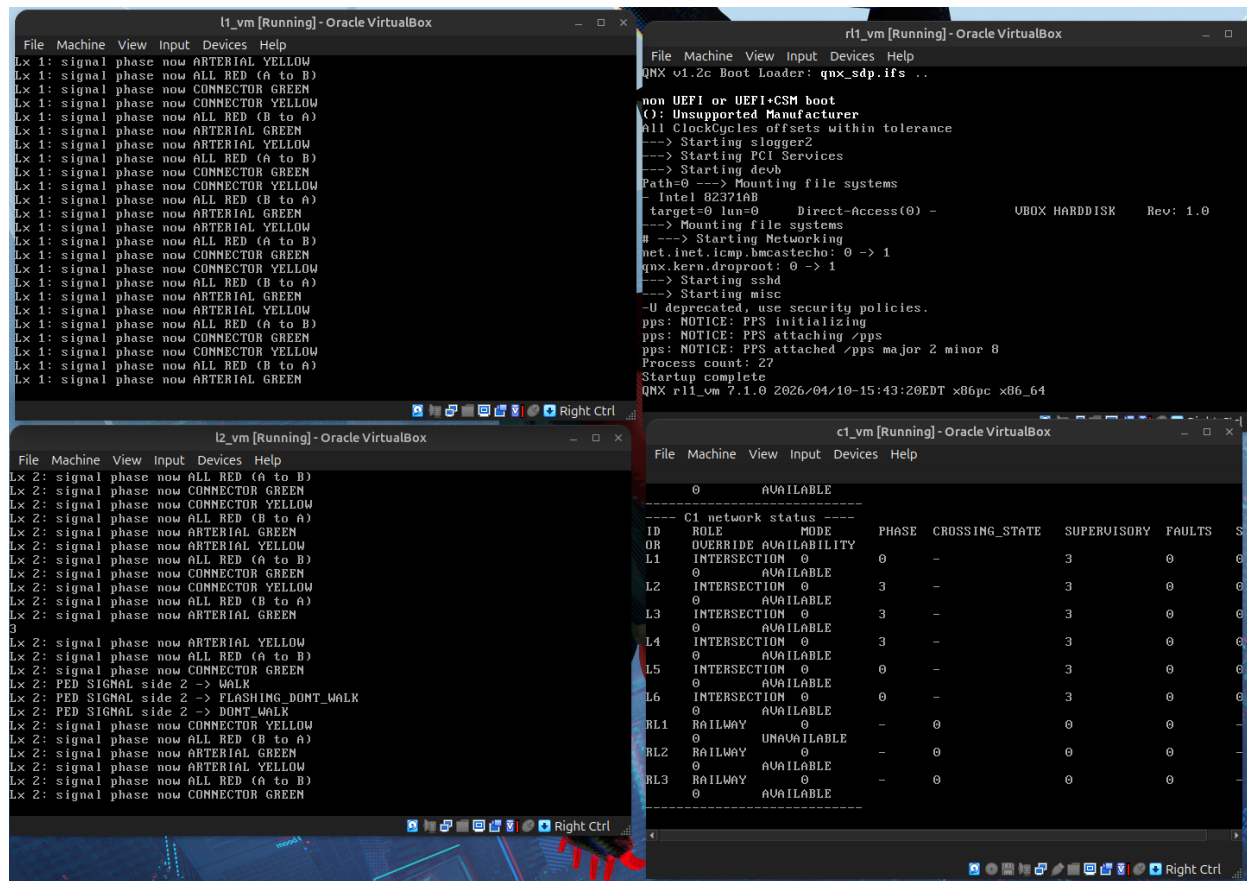


Figure 7. Full 90-second deterministic peak fixed-timing phase cycle (PEAK_FIXED)

7.1.2 Sensor-Actuated Vehicle Detection & Anti-Starvation Cap (UC-01, TL-02)

To minimize off-peak delays, the controller transitions to `MODE_OFF_PEAK_SENSOR` upon supervisory command (`MSG_SET_MODE`) or scheduled time-of-day trigger (DP-01). In this mode, signal allocations respond dynamically to simulated road loop sensors:

- **Major Route Resting Policy:** In the absence of conflicting demand, the controller rests indefinitely on `PHASE_ARTERIAL_GREEN`, eliminating unnecessary vehicle stops on the primary thoroughfare.
- **Side-Street Actuation:** Pressing `c` on the `lx_sensor` console asserts connector vehicle presence (`SENSOR_CONNECTOR_DEMAND`). Before yielding the right-of-way, the controller enforces a mandatory minimum green interval of 8.0 s (`LX_MIN_GREEN_MS`) on the arterial, followed by standard 4.0 s yellow and 2.0 s all-red clearance.
- **Dynamic Green Extension:** While connector green is active, successive car arrivals (pressing `c`) dynamically extend the green phase by 4.0 s per arrival (`LX_EXTENSION_MS = 4000 ms`), accommodating platoons.
- **Anti-Starvation Maximum Green Cap (Edge Invariant):** Continuous vehicle arrivals on a minor approach cannot permanently starve the arterial road. The FSM strictly bounds the maximum allowable green duration to 40.0 s (`LX_MAX_GREEN_MS = 40000 ms`). When the 40.0 s ceiling expires, the controller unconditionally forces a transition to yellow, serving the arterial queue regardless of active side-street calls.

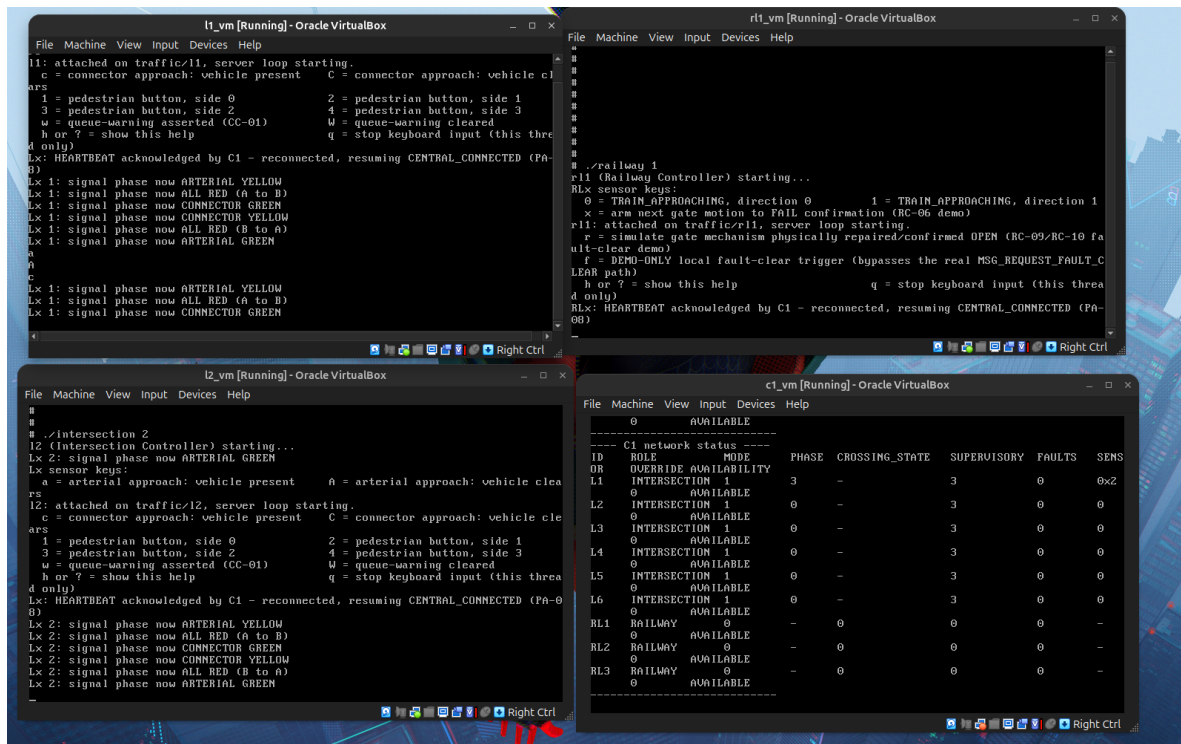


Figure 8. Sensor-actuated vehicle detection, dynamic green extension, and anti-starvation cap (OFF_PEAK_SENSOR) (1)

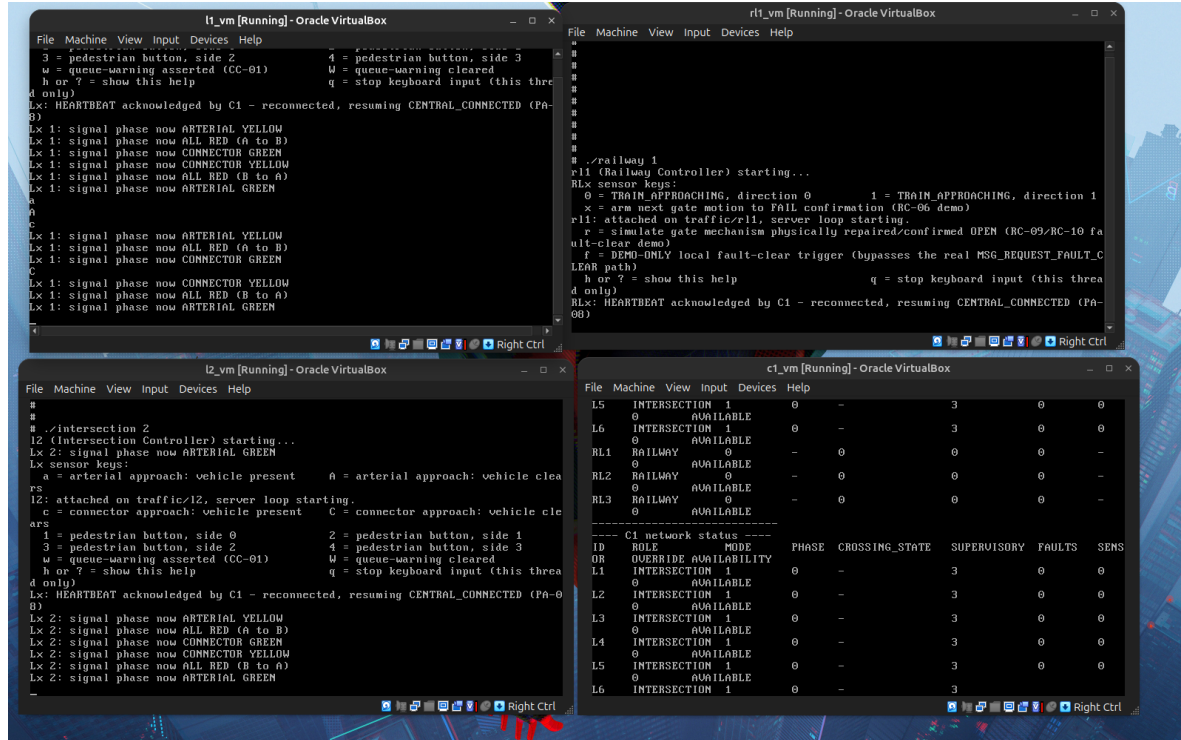


Figure 9. Sensor-actuated vehicle detection, dynamic green extension, and anti-starvation cap (OFF_PEAK_SENSOR) (2)

Figure 8 (Vehicle Demand Present): Lowercase key c simulates a car arriving on the connector approach. Controller L1 services the demand by safely transitioning from arterial to CONNECTOR GREEN. Concurrently, Central's monitoring matrix (c1_vm) reflects the active call, registering MODE = 1 (OFF_PEAK_SENSOR), PHASE = 3 (CONNECTOR GREEN), and bitmask SENSOR = 0x2 (SENSOR_CONNECTOR_DEMAND).

Figure 9 (Vehicle Demand Cleared): Uppercase key C simulates vehicle departure. With side-street demand extinguished, L1 executes standard clearance (YELLOW → ALL RED) and returns to resting on ARTERIAL GREEN. Central's telemetry immediately confirms state recovery, recording PHASE = 0 and SENSOR = 0. This confirms dynamic demand-responsive allocation and zero-latency telemetry tracking.

7.1.3 Pedestrian Pushbutton Latching & Clearance Cycle (UC-02)

Crosswalk pushbuttons simulated via keys 1–4 trigger the pedestrian service state machine (SENSOR_PED_LATCHED_SIDE_*):

- **Demand Latching and Press Coalescing:** Once a pedestrian call is registered, subsequent rapid button presses coalesce into the active latch without resetting timers or spawning redundant cycles.
- **Protected Walk Interval:** Pedestrians are served concurrently with the parallel vehicle green, displaying WALK for 6.0 s (LX_WALK_MS).
- **Flashing Clearance Interval:** The WALK display is succeeded by a mandatory 4.0 s flashing clearance (LX_FLASHING_DONT_WALK_MS), warning pedestrians that the crossing window is ending before vehicle amber begins.
- **Pre-emption Memory Retention (Safety Invariant):** If a higher-priority railway pre-emption occurs while a pedestrian call is latched, the pedestrian demand is preserved in memory and guaranteed service at the earliest safe boundary following crossing reopening.

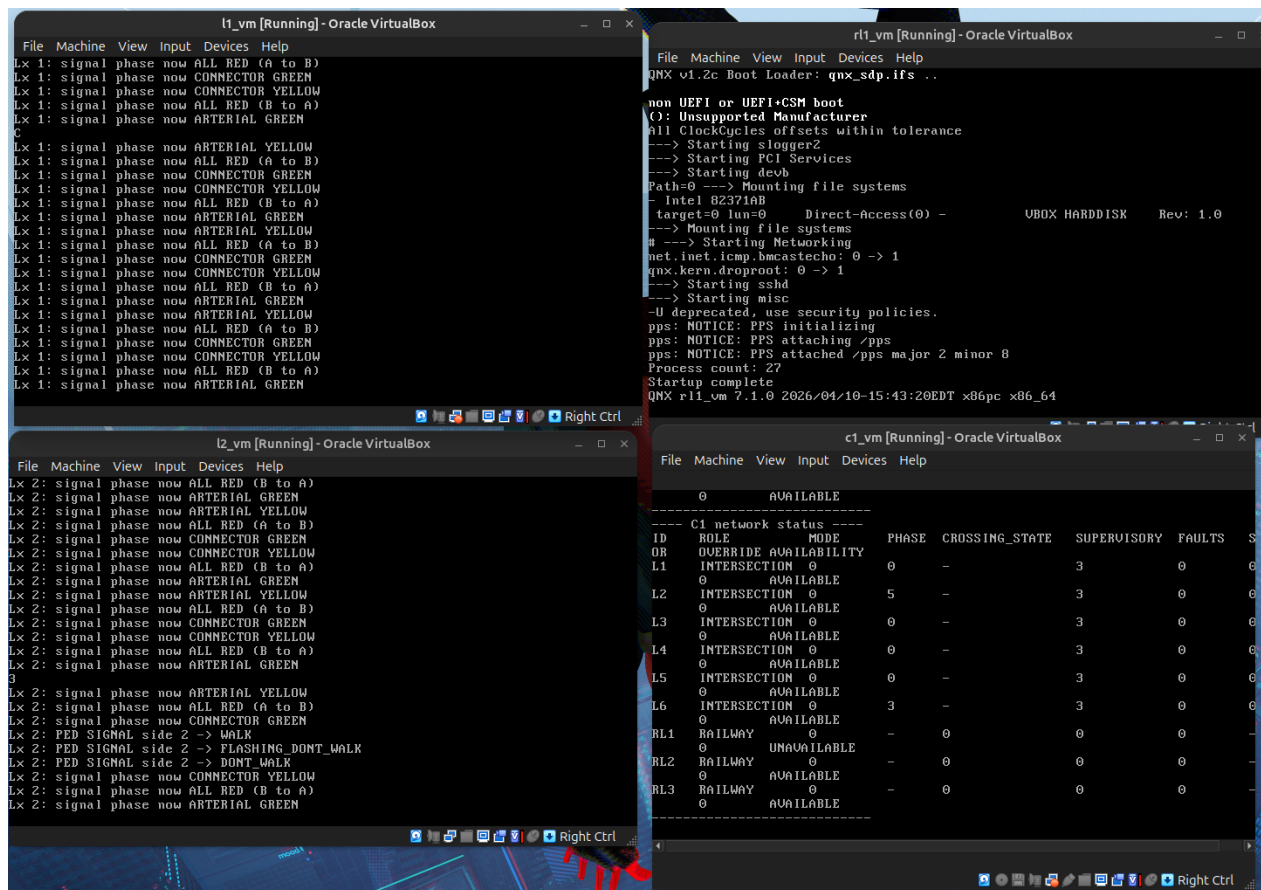


Figure 10. Pedestrian pushbutton latching, protected walk interval, and flashing clearance cycle

Figure 10 verifies pedestrian crossing demand servicing and the three-stage clearance interval (UC-02) on controller L2. Upon registering crosswalk pushbutton 3 (SENSOR_PED_LATCHED_SIDE_2), the controller safely cycles through arterial clearance into the compatible connector stage.

As evidenced in the terminal trace, the pedestrian signal executes the full protected sequence: granting a 6.0 s WALK interval (LX_WALK_MS), followed by a mandatory 4.0 s clearance warning (FLASHING_DONT_WALK), before reverting to solid DONT_WALK. Crucially, vehicular amber (CONNECTOR YELLOW) is initiated only after the crosswalk is fully secured in DONT_WALK, proving strict mathematical isolation between conflicting pedestrian and vehicular phases.

7.2 Railway Protection and Closure Traffic

7.2.1 The 45-Second Approach Warning & Hardware Gate Interlock (UC-04, RC-01–RC-04)

Railway level crossings (RL1–RL3) operate under a strict 45-second approach warning budget (RLX_WARNING_BUDGET_MS = 45000 ms) to protect the double-track rail corridor. Pressing 0 or 1 on `rlx_sensor` simulates track circuit arrival (TRAIN_APPROACHING):

- **Immediate Pre-warning (0-5 s):** Audible bells and red warning flashers activate immediately; the crossing enters `CROSSING_WARNING`. Simultaneously, RLx broadcasts `MSG_CROSSING_STATUS` over Qnet to adjacent intersections (L1/L2) and C1.
- **Boom Gate Lowering (5-13 s):** After a 5.0 s warning buffer, boom gates are commanded down, simulating 8.0 s of mechanical movement (RLX_GATE_MOTION_MS = 8000 ms).
- **The Confirmation-Before-Proceed Invariant (RC-04):** Train approach signals remain locked at STOP. Signals transition to PROCEED (GREEN) if and only if physical limit switches assert `GATES_CONFIRMED_CLOSED`. Trains never receive a proceeding signal based merely on a timer.

```

rl1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
# cd tmp
# ./
keep_files    railway
# ./railway 1
rl1 (Railway Controller) starting...
RLx sensor keys:
  0 = TRAIN_APPROACHING, direction 0          1 = TRAIN_APPROACHING, direction 1
  x = arm next gate motion to FAIL confirmation (RC-06 demo)
rl1: attached on traffic/rl1, server loop starting.
  r = simulate gate mechanism physically repaired/confirmed OPEN (RC-09/RC-10 f
ult-clear demo)
  f = DEMO-ONLY local fault-clear trigger (bypasses the real MSG_REQUEST_FAULT
LEAR path)
  h or ? = show this help                      q = stop keyboard input (this thre
d only)
RLx: HEARTBEAT acknowledged by C1 - reconnected, resuming CENTRAL_CONNECTED (P
0B)
0
RLx: flashers ON (train approaching, direction 0)
RLx: commanding gates DOWN (simulated motion, 3000 ms)
RLx: train signal PROCEED for direction 0 (gates confirmed closed)
RLx: all train signals -> STOP (crossing reopening)
RLx: commanding gates UP (simulated motion, 3000 ms)
RLx: flashers OFF (gates confirmed open)

```

Figure 11. 45-second approach warning, boom gate closure, and confirmation-before-proceed train signal interlock (1)

```

0 AVAILABLE
-----
---- C1 network status ----
ID  ROLE  MODE  PHASE  CROSSING_STATE  SUPERVISORY  FAULTS  S
OR  OVERRIDE AVAILABILITY
L1  INTERSECTION  0  0  -  3  0  0
    0  AVAILABLE
L2  INTERSECTION  0  2  -  3  0  0
    0  AVAILABLE
L3  INTERSECTION  0  3  -  3  0  0
    0  AVAILABLE
L4  INTERSECTION  0  1  -  3  0  0
    0  AVAILABLE
L5  INTERSECTION  0  0  -  3  0  0
    0  AVAILABLE
L6  INTERSECTION  0  0  -  3  0  0
    0  AVAILABLE
RL1 RAILWAY  0  -  0  0  0  -
    0  AVAILABLE
RL2 RAILWAY  0  -  0  0  0  0  -
    0  AVAILABLE
RL3 RAILWAY  0  -  0  0  0  0  -
    0  AVAILABLE
-----

```

Figure 12. 45-second approach warning, boom gate closure, and confirmation-before-proceed train signal interlock (2)

Figure 11-12 verifies the 45-second approach warning sequence and gate confirmation interlock (UC-04, RC-01–RC-04). When an approaching train is detected on Track 0, controller RL1 immediately activates audible warnings and red flashers (`CROSSING_WARNING`). Following a 5.0 s warning buffer, boom gates are commanded down. Crucially, train signals remain locked at `STOP` until physical limit switches confirm `GATES_CONFIRMED_CLOSED` at $t=8s$, at which point the signal transitions to `PROCEED` for direction 0. Following train clearance, train signals revert to `STOP`, boom gates are commanded open, and flashers extinguish only after gates confirm fully open (`CROSSING_OPEN`).

7.2.2 Adjacent Intersection Direct Traffic Suppression (CC-02, RLx → Lx)

Upon receiving `MSG_CROSSING_STATUS(CROSSING_CLOSED)`, adjacent intersection controllers (L1/L2) autonomously engage in pre-emption suppression:

- **Connector Approach Suppression:** Movement phases leading toward the railway tracks are immediately driven to `RED` via standard yellow and all-red clearance, preventing vehicles from queuing onto the tracks.
- **Non-Conflicting Arterial Flow:** Traffic movements parallel to the tracks remain operational on green, preventing unnecessary network-wide gridlock during train crossings.

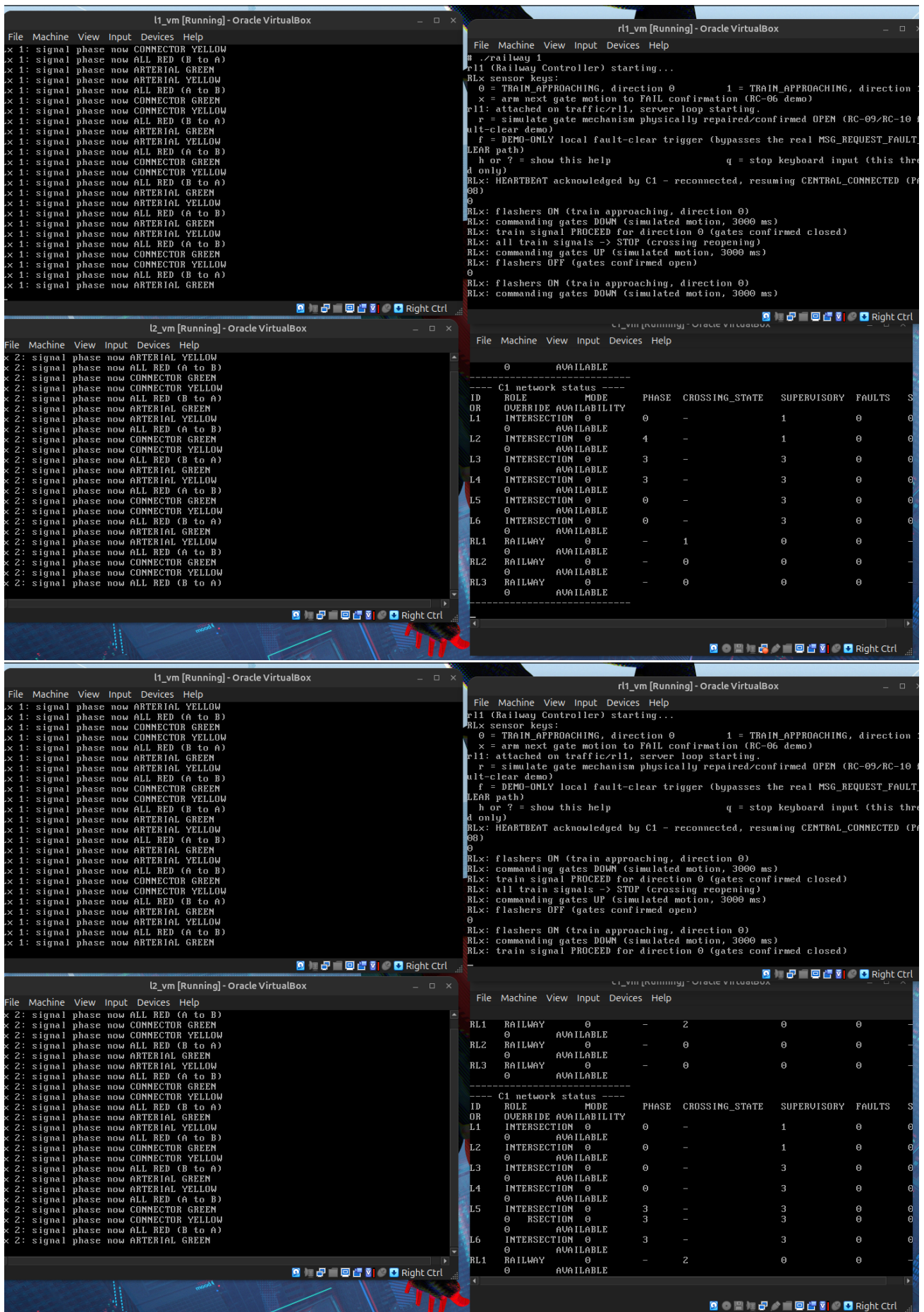
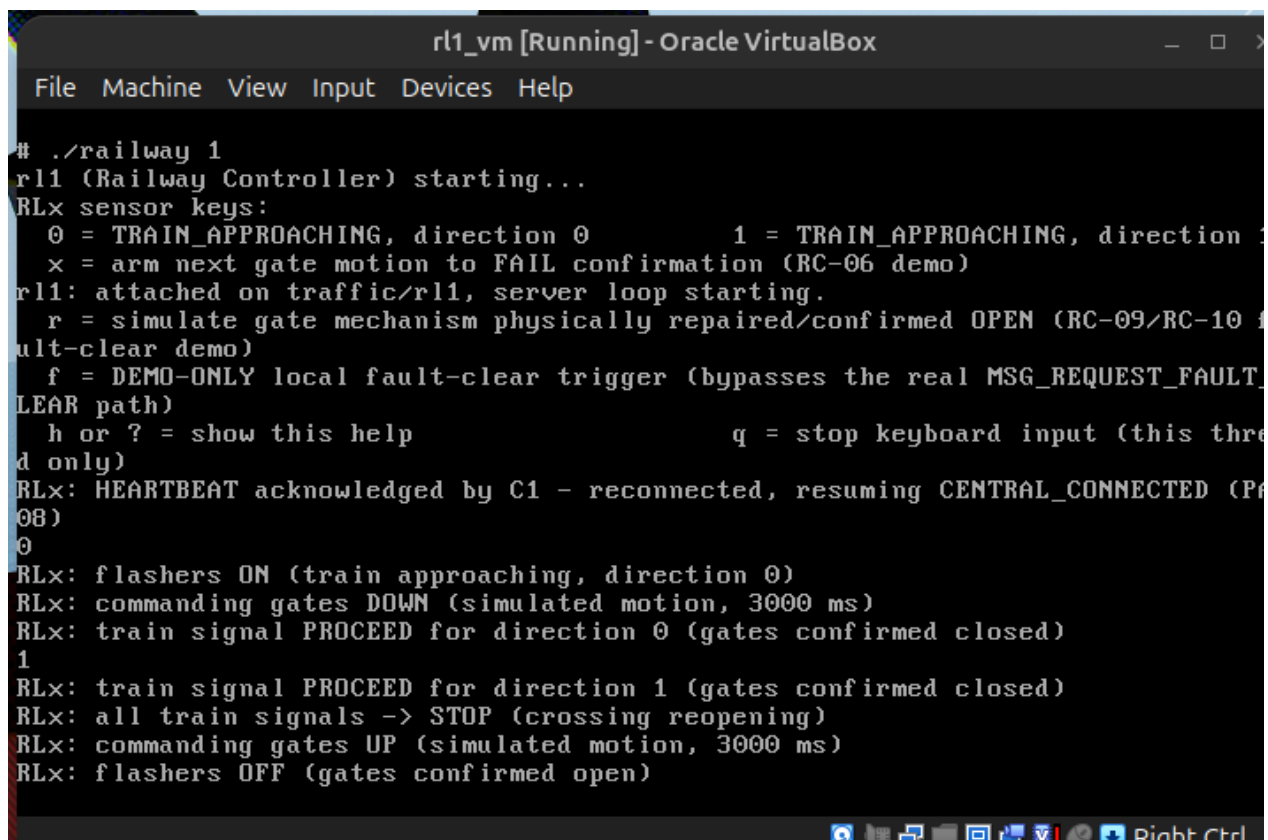


Figure 13. Adjacent intersection direct traffic suppression (holding connector red toward tracks)

Figure 13 demonstrates direct peer-to-peer traffic suppression at adjacent intersection L1 during railway closure (CC-02, RLx → Lx). Upon receiving `MSG_CROSSING_STATUS(CROSSING_CLOSED)` from peer controller RL1, L1 immediately terminates connector stage green via standard yellow and all-red clearance. As evidenced in the terminal trace, during the entire duration of the railway pre-emption, connector green is suppressed; the controller loops continuously from `ALL RED (A to B)` directly into `ARTERIAL GREEN`. This prevents dangerous vehicle queue spillbacks across the railway tracks while allowing non-conflicting parallel arterial traffic to progress safely.

7.2.3 Dual-Track Second Train Handling (UC-04 Alt 8.1)

If a second train approaches Track 1 while Track 0 is actively occupied (`TRAIN_PRESENT`), the railway FSM dynamically extends the closure window. Boom gates remain locked down without cycling, eliminating vehicle entrapment hazards between closely spaced trains.



```

rl1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help

# ./railway 1
rl1 (Railway Controller) starting...
RLx sensor keys:
  0 = TRAIN_APPROACHING, direction 0          1 = TRAIN_APPROACHING, direction 1
  x = arm next gate motion to FAIL confirmation (RC-06 demo)
rl1: attached on traffic/rl1, server loop starting.
  r = simulate gate mechanism physically repaired/confirmed OPEN (RC-09/RC-10 f
ult-clear demo)
  f = DEMO-ONLY local fault-clear trigger (bypasses the real MSG_REQUEST_FAULT
LEAR path)
  h or ? = show this help                    q = stop keyboard input (this thro
d only)
RLx: HEARTBEAT acknowledged by C1 - reconnected, resuming CENTRAL_CONNECTED (P
08)
0
RLx: flashers ON (train approaching, direction 0)
RLx: commanding gates DOWN (simulated motion, 3000 ms)
RLx: train signal PROCEED for direction 0 (gates confirmed closed)
1
RLx: train signal PROCEED for direction 1 (gates confirmed closed)
RLx: all train signals -> STOP (crossing reopening)
RLx: commanding gates UP (simulated motion, 3000 ms)
RLx: flashers OFF (gates confirmed open)

```

Figure 14. Dual-track second train occupancy extension without barrier reopening

Figure 14 verifies dual-track second train handling (UC-04 Alt 8.1, RC-04). While Train 0 was actively occupying the crossing (`TRAIN_PRESENT`), a second train was detected on opposing Track 1 (1 key). Rather than initiating gate reopening at the conclusion of Train 0's occupancy window ($t = 48$ s), the controller dynamically extended the barrier closure window, maintaining `GATES_CONFIRMED_CLOSED`. Boom gates remained locked down without cycling, eliminating vehicle entrapment hazards. The crossing reopened only after both occupancy windows expired, ensuring complete protection for multi-train corridor events.

7.2.4 Post-Crossing Congestion Management & Queue Drain Phase (CC-01, UC-05)

Vehicles held behind closed boom gates form long congestion queues. To clear this bottleneck upon crossing reopening:

- **Queue Warning Assertion:** Advance loop sensors detect backed-up traffic, asserting `SENSOR_QUEUE_WARNING` (w key).
- **Dynamic Queue Drain Green Flush:** When boom gates confirm fully open (`CROSSING_OPEN`), the adjacent intersection activates a dedicated **Queue Drain Phase**, granting an immediate green flush to the connector approach.
- **Incremental Extension & Bound:** The drain phase extends in 4.0 s increments up to a 60.0 s ceiling (`LX_DRAIN_MAX_EXTENSION_MS = 60000 ms`). When the queue warning clears (w key) or the 60 s cap expires, the intersection smoothly transitions back to normal cycling.

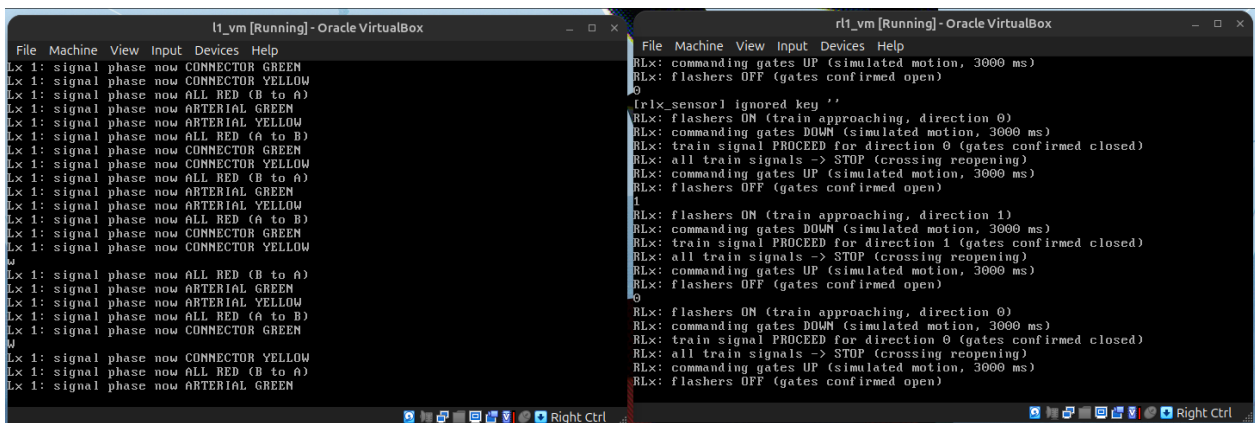


Figure 15. Post-crossing congestion queue detection and dynamic green flush drain phase

Figure 15 verifies post-crossing congestion control and the dynamic Queue Drain Phase (CC-01, UC-05). Advance vehicle sensors detected congestion queued behind the railway barrier, asserting `SENSOR_QUEUE_WARNING` (w key). Upon receiving `CROSSING_OPEN` from RL1, controller L1 initiated a dedicated Queue Drain Phase on `CONNECTOR GREEN`. Instead of terminating at the standard 30-second ceiling, connector green was dynamically extended in 4.0 s increments to flush queued traffic. Once uppercase w signaled that the queue had cleared, the controller safely concluded the drain phase via standard yellow and all-red clearance.

7.2.5 Boom Gate Failure Latching & Safe Recovery (UC-06, RC-06, RC-09)

Arming a mechanical fault (x key) simulates a gate obstruction during closure:

- **Fail-Safe Signal Hold:** If gate confirmation is missing after 8.0 s of motion, the crossing trips `FAULT_GATE_CONFIRM_MISSING`. Train signals are locked at `STOP`, flashers remain active, and `MSG_FAULT_REPORT` is dispatched to Central.
- **Recovery Re-Verification:** An operator command from Central (`MSG_REQUEST_FAULT_CLEAR`) is rejected with `NACK_REASON_FAULT_ACTIVE` if live gate sensors detect physical obstruction (`gates_confirmed_open()`), enforcing on-site physical verification before fault clearance.

```

rl1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1
RLx: flashers ON (train approaching, direction 1)
RLx: commanding gates DOWN (simulated motion, 3000 ms)
RLx: train signal PROCEED for direction 1 (gates confirmed closed)
RLx: all train signals -> STOP (crossing reopening)
RLx: commanding gates UP (simulated motion, 3000 ms)
RLx: flashers OFF (gates confirmed open)
0
RLx: flashers ON (train approaching, direction 0)
RLx: commanding gates DOWN (simulated motion, 3000 ms)
RLx: train signal PROCEED for direction 0 (gates confirmed closed)
RLx: all train signals -> STOP (crossing reopening)
RLx: commanding gates UP (simulated motion, 3000 ms)
RLx: flashers OFF (gates confirmed open)
x
[DEMO] Next gate motion armed to fail confirmation (RC-06 fault path)
0
RLx: flashers ON (train approaching, direction 0)
RLx: commanding gates DOWN (simulated motion, 3000 ms)
RLx: gate FAILED TO CONFIRM (simulated fault) - rlxfsm.c's own deadline will raise FAULT_GATE_CONFIRM_MISSING
RLx: commanding gates DOWN (simulated motion, 3000 ms)
RLx: FAULT latched (GATE_CONFIRM_MISSING, bit 0x1) - holding STOP on all train signals, commanding gates DOWN

```

Figure 16. Boom gate obstruction failure latching, train stop signal, and recovery verification rejection

Figure 16 proves fail-safe hardware fault latching and autonomous safe-state enforcement (UC-06, RC-06, RC-09). A mechanical obstruction was simulated on boom gate closure using key x. When Train 0 approached, the boom gates moved down but failed to assert position limit confirmation. Upon expiration of the confirmation deadline, controller RL1 tripped `FAULT_GATE_CONFIRM_MISSING` (bit 0x1) and latched into `CROSSING_FAULT`. Crucially, train signals were permanently locked at `STOP`, preventing train entry into an unconfirmed crossing, while boom gates were held down to protect motorists. An asynchronous `MSG_FAULT_REPORT` was forwarded to Central without blocking local safety execution.

7.3 Central Monitoring, Commands, and Local Autonomy

7.3.1 Central Telemetry Aggregation & Real-Time HMI (UC-09)

The Central supervisory controller (`c1`) aggregates 1 Hz telemetry from all 10 nodes, displaying a real-time system matrix in `c_hmi.c`. The dashboard tracks node role, operational mode, active signal phase, crossing state, supervisory priority, sensor bitmasks, active faults, and link availability, providing immediate situational awareness to control-room operators.

ID	ROLE	MODE	PHASE	CROSSING_STATE	SUPERVISORY	FAULTS	SENS
L1	INTERSECTION	0	0	-	1	0	0
L2	INTERSECTION	0	0	-	1	0	0
L3	INTERSECTION	0	0	-	3	0	0
L4	INTERSECTION	0	2	-	3	0	0
L5	INTERSECTION	0	0	-	3	0	0
L6	INTERSECTION	0	0	-	3	0	0
RL1	RAILWAY	0	-	3	0	0x1	-
RL2	RAILWAY	0	-	0	0	0	-
RL3	RAILWAY	0	-	0	0	0	-

Figure 17. Central 1 Hz real-time network telemetry monitoring matrix

Figure 17 displays Central's 1 Hz aggregated network telemetry dashboard (`c_hmi.c`), satisfying requirement UC-09. Driven by periodic 1-second pulse timers (`IPC_PULSE_HEARTBEAT_TICK`), Central collects embedded telemetry payloads from all 10 distributed QNX nodes. The matrix provides real-time supervisory tracking of controller roles, active operating modes, signal phases, railway barrier states, 4-tier supervisory priority levels, sensor bitmasks, and node availability. Nodes reporting active heartbeats are flagged as `AVAILABLE`, confirming live end-to-end *Qnet* connectivity without polling overhead.

7.3.2 Priority Vehicle & Dignitary Clear-Route Override (UC-08, PA-11)

Central operators can clear priority corridors for emergency vehicles or visiting dignitaries using console key (`handle_request_override()`):

- **Command Dispatch:** Central sends `MSG_REQUEST_OVERRIDE` specifying target `Lx`, movement axis (arterial vs. connector), and duration (capped at 300,000 ms / 5 minutes per PA-11).
- **Continuous Green Hold:** The target intersection overrides normal cyclic stage timers, forcing and holding green on the requested movement.
- **Controlled Lifecycle:** Overrides can be extended via key `r` (`MSG_RENEW_OVERRIDE`) or terminated early via key `c` (`MSG_CANCEL_OVERRIDE`). Early cancellation executes safe yellow and all-red clearance intervals without abrupt light cutoffs.

```

c1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
[2026-09-19 11:30:26] Controller 4 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:30:27] Controller 3 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:30:27] Controller 8 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:30:28] Controller 3 reconnected (PA-08)
[2026-09-19 11:30:28] Controller 8 reconnected (PA-08)
[2026-09-19 11:30:28] Controller 4 reconnected (PA-08)
[2026-09-19 11:43:34] Controller 7 reconnected (PA-08)
[2026-09-19 11:50:30] Controller 7 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:50:32] Controller 7 reconnected (PA-08)
[2026-09-19 12:09:49] FAULT_REPORT from 7: fault_code=0x00000001 severity=1 detail="RLx fault - see fault_code bitmask"
[2026-09-19 12:15:26] Operator: REQUEST_OVERRIDE(target=2, movement=1, duration_ms=22) submitted
[2026-09-19 12:15:26] C1: REQUEST_OVERRIDE to 2 -> NACK reason=RAILWAY_CONFLICT
[2026-09-19 12:17:24] Operator: REQUEST_OVERRIDE(target=1, movement=1, duration_ms=222222) rejected by Central pre-check, reason=INVALID_DURATION - not forwarded to the controller
[2026-09-19 12:27:08] Operator: REQUEST_OVERRIDE(target=1, movement=0, duration_ms=30000) submitted
[2026-09-19 12:27:08] C1: REQUEST_OVERRIDE to 1 -> ACK
# _

```

Figure 18. Priority vehicle / dignitary clear-route override dispatch and lifecycle management

Figure 18 verifies local safety arbitration and priority hierarchy enforcement (UC-08 Alt 3.2, PA-09). During an active level-crossing closure on corridor RL1, a supervisory override request was submitted to force green on adjacent connector approach L2 (*movement=1*). In strict accordance with the system's 4-tier supervisory hierarchy, where Railway Pre-emption strictly supersedes Central Overrides, controller L2 evaluated the conflict locally and rejected the command, returning `RESULT_NACK` with reason `RAILWAY_CONFLICT`. This proves that remote supervisory commands cannot bypass autonomous local track-protection interlocks.

7.3.3 The 4-Tier Supervisory Safety Hierarchy & Interlock Rejections (NACKs)

Local controller autonomy dictates that safety invariants unconditionally supersede Central supervisory requests:

- **Pedestrian Deferral (`RESULT_ACK_PENDING`):** If an override is commanded while pedestrian clearance is active, the intersection defers activation (`RESULT_ACK_PENDING`), completing the full 4.0 s flashing clearance before granting green to vehicles (UC-08 Alt 3.1).
- **Railway Superiority (`NACK_REASON_RAILWAY_CONFLICT`):** If Central commands an override toward an active or closing railway crossing, the local controller rejects the request with `RESULT_NACK` (UC-08 Alt 3.2). Railway corridor safety strictly overrules emergency vehicle priority.
- **Duration Bounds:** Requests exceeding 300,000 ms are rejected immediately with `NACK_REASON_INVALID_DURATION` (PA-11).

Figure 19 below demonstrates defensive input validation and supervisory parameter bounding (UC-08 Alt 3.3, PA-11). An operator attempted to dispatch a clear-route override specifying an excessive duration of 2,222,222 ms. Central's command pre-check layer (`c_mode_eng_validate_override_request()`) evaluated the payload against the mandatory 300,000 ms (5-minute) safety ceiling defined by rule PA-11. The request was immediately rejected with reason=`INVALID_DURATION` and dropped before reaching the *Qnet* client queue, preventing unbounded overrides or network transmission of invalid requests.

```

c1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
[2026-09-19 11:30:26] Controller 4 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:30:27] Controller 3 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:30:27] Controller 8 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:30:28] Controller 3 reconnected (PA-08)
[2026-09-19 11:30:28] Controller 8 reconnected (PA-08)
[2026-09-19 11:30:28] Controller 4 reconnected (PA-08)
[2026-09-19 11:43:34] Controller 7 reconnected (PA-08)
[2026-09-19 11:50:30] Controller 7 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 11:50:32] Controller 7 reconnected (PA-08)
[2026-09-19 12:09:49] FAULT_REPORT from 7: fault_code=0x00000001 severity=1 detail="RLx fault - see fault_code bitmask"
[2026-09-19 12:15:26] Operator: REQUEST_OVERRIDE(target=2, movement=1, duration_ms=22) submitted
[2026-09-19 12:15:26] C1: REQUEST_OVERRIDE to 2 -> NACK reason=RAILWAY_CONFLICT
[2026-09-19 12:17:24] Operator: REQUEST_OVERRIDE(target=1, movement=1, duration_ms=222222) rejected by Central pre-check, reason=INVALID_DURATION - not forwarded to the controller
[2026-09-19 12:27:08] Operator: REQUEST_OVERRIDE(target=1, movement=0, duration_ms=30000) submitted
[2026-09-19 12:27:08] C1: REQUEST_OVERRIDE to 1 -> ACK
# _

```

Figure 19. Local safety arbitration: VIP override deferred during pedestrian walk and rejected during railway preemption

7.3.4 Arterial Green-Wave Coordination & Mode Management (UC-03, UC-07)

Operators can broadcast coordinated timing profiles across arterial roads R1 (L1–L3) and R2 (L4–L6) using key `t` (`MSG_SET_TIMING_PROFILE`), establishing green-wave offsets based on physical distance and vehicle progression speeds. Supervisory mode changes (`MSG_SET_MODE`) are deferred (`RESULT_ACK_PENDING`) and applied strictly at safe cycle boundaries (TL-04, DP-01).

```

c1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
ms=22) submitted
[2026-09-19 12:15:26] C1: REQUEST_OVERRIDE to 2 -> NACK reason=RAILWAY_CONFLICT
[2026-09-19 12:17:24] Operator: REQUEST_OVERRIDE(target=1, movement=1, duration_ms=222222) rejected by Central pre-check, reason=INVALID_DURATION - not forwarded to the controller
[2026-09-19 12:27:08] Operator: REQUEST_OVERRIDE(target=1, movement=0, duration_ms=30000) submitted
[2026-09-19 12:27:08] C1: REQUEST_OVERRIDE to 1 -> ACK
[2026-09-19 12:30:39] Operator: demo hour forced to 8 -> schedule implies mode=PEAK_FIXED, broadcasting to all Lx
[2026-09-19 12:30:39] C1: SET_MODE to 1 -> ACK_PENDING
[2026-09-19 12:30:39] C1: SET_MODE to 2 -> ACK_PENDING
[2026-09-19 12:30:39] C1: SET_MODE to 3 -> ACK_PENDING
[2026-09-19 12:30:39] C1: SET_MODE to 4 -> ACK_PENDING
[2026-09-19 12:30:39] C1: SET_MODE to 5 -> ACK_PENDING
[2026-09-19 12:30:39] C1: SET_MODE to 6 -> ACK_PENDING
[2026-09-19 12:31:08] Controller 1 marked UNAVAILABLE - missed 3 consecutive heartbeats (PA-07)
[2026-09-19 12:31:11] Controller 1 reconnected (PA-08)
[2026-09-19 12:33:08] Operator: SET_TIMING_PROFILE broadcast (chain=R1, profile_id=1) submitted
[2026-09-19 12:33:08] C1: SET_TIMING_PROFILE to 1 -> ACK
[2026-09-19 12:33:08] C1: SET_TIMING_PROFILE to 3 -> ACK
[2026-09-19 12:33:08] C1: SET_TIMING_PROFILE to 5 -> ACK
#

```

Figure 20. Coordinated arterial green-wave timing profile broadcast and safe-boundary mode switching

Figure 20 demonstrates arterial coordination and mode management (UC-03, UC-07). Using console command `t`, Central broadcast `MSG_SET_TIMING_PROFILE` to arterial chain 1 (L1–L3 along Road R1), distributing green-wave offsets calculated from intersection spacing and design travel speeds. Field intersections acknowledged the coordination parameters synchronously with `RESULT_ACK`. Furthermore, mode changes (`MSG_SET_MODE`) returned `RESULT_ACK_PENDING`, verifying that transitions between `PEAK_FIXED` and `OFF_PEAK_SENSOR` are deferred to safe inter-phase clearance boundaries (TL-04, DP-01)

7.3.5 Communication Loss (LINK_DEGRADED_LOCAL) & Instant State Sync (UC-10, PA-07, PA-08)

```

l1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT acknowledged by C1 - reconnected, resuming CENTRAL_CONNECTED (PA-08)
Lx 1: signal phase now ARTERIAL YELLOW
Lx 1: signal phase now ALL RED (A to B)
Lx 1: signal phase now CONNECTOR GREEN
Lx 1: signal phase now CONNECTOR YELLOW
Lx 1: signal phase now ALL RED (B to A)
Lx 1: signal phase now ARTERIAL GREEN
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: 3 consecutive HEARTBEATS unacknowledged - entering DEGRADED_LOCAL (PA-07)

l1_vm [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx 1: signal phase now CONNECTOR YELLOW
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx 1: signal phase now ALL RED (B to A)
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx 1: signal phase now ARTERIAL GREEN
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send
Lx: HEARTBEAT to 0 failed to send

```

Figure 21. Heartbeat loss fail-safe degradation (LINK_DEGRADED_LOCAL) and instant reconnection resynchronization

The system guarantees complete fault isolation and resilience against network partitioning:

- **Local Autonomy Under Link Failure (PA-07):** If Central terminates or network connectivity fails, field controllers record 3 consecutive missed heartbeats (3 seconds) and enter

`LINK_DEGRADED_LOCAL`. Controllers continue autonomous traffic sequencing, vehicle actuation, and railway pre-emption without interruption.

- **Zero-Handshake Instant Resynchronization (PA-08):** When Central restarts, the first incoming 1 Hz `MSG_HEARTBEAT` delivers complete controller state telemetry, instantly restoring Central's monitoring matrix without requiring node restarts or manual resync sequences

Figure 21 demonstrates the distributed fault tolerance and local autonomy mechanisms (PA-07, PA-08, UC-10). When communication with the Central controller is severed (Central process termination), node L1 records three consecutive unacknowledged 1 Hz heartbeats and autonomously enters `LINK_DEGRADED_LOCAL` at $t=3s$. Crucially, as evidenced in the terminal trace, heartbeat transmission failures do not block or stall the intersection: the 100 ms timer-driven sequencer continues running without interruption, cycling phases through `CONNECTOR YELLOW`, `ALL RED (B to A)`, and `ARTERIAL GREEN`, while continuing to service local sensor calls (`c/c`). Outbound IPC failures are safely handled by the dedicated non-blocking client worker thread. When Central restarts, the first successful 1 Hz heartbeat acknowledgment immediately restores the node to `LINK_CENTRAL_CONNECTED` (PA-08) with zero-handshake state recovery, requiring no manual intervention or node restart.

8. Verification and Test Results

Verification covers normal operation, timing boundaries, conflicting requests, fault responses, and recovery. The test-plan files define the inputs and expected results; this section records **what was executed and observed**, followed by timing evidence and unresolved limitations. A test is marked **Pass** only when its observed behavior can be compared with a stated criterion and supporting console output, log, or measurement.

8.1 Test Environment and Method

Environment and setup

- Testing used the **ten-VM QNX deployment** described in **Section 2.3**, with communication between controllers over *Qnet*. Each case started with the controllers required by its preconditions; standalone and failure cases deliberately ran with selected controllers absent or stopped.
- **Section 2.3** is the reference for binary deployment, `TRAFFIC_NODE_MAP`, startup order, and initial connectivity checks.

Test method

- Functional and conflict cases used C1 operator commands and simulated sensor keys at Lx and RLx. Failure and recovery cases additionally interrupted or restarted the relevant controller, then checked how the remaining nodes responded.
- Expected behavior was compared with the output in each controller's terminal, C1's network-status display, and relevant entries in `central_log.txt`. **Lx and RLx terminal output is not stored in `central_log.txt`. Section 8.3 states the measurement method used for each timing result.**
- A runtime **Pass** requires the test precondition to be established and the observed result to match the expected result. Checks completed by code review are counted separately. Cases lacking a required tool, fault precondition, or recorded execution are marked **Skip**.

Result accounting

- Across the seven test-plan files, **232 cases** are reported: **201 runtime Pass**, **6 code-review Pass**, **25 Skip**, and **0 Fail**.

- The 25 Skip cases comprise 12 requiring `test_client`, nine requiring an Lx `FAULT_SAFE` precondition that was not established, three requiring a debugger, and one `kill -STOP/kill -CONT` experiment that was not executed and recorded. Representative results appear in **Section 8.2**; unresolved conditions appear in **Section 8.4**.

8.2 Requirements and Scenario Coverage

The complete test procedures, preconditions, and expected outcomes are documented in the project's [test plan](#). **Table 15** reports representative results across normal operation, boundary values, conflicting requests, concurrent inputs, faults, and recovery. Timing measurements are detailed in **Section 8.3**.

Table 15. Representative test results

Test ID	Requirement / UC	Input and precondition	Expected result	Observed result	Status
TC-UC01-1	UC-01: fixed vehicle-signal cycle	Start L1 in <code>PEAK_FIXED</code> ; provide no sensor input for one full cycle.	Phases follow arterial green → yellow → all red → connector green → yellow → all red, then repeat.	L1 terminal showed the phases in that order and returned to arterial green; no phase was skipped.	Pass — runtime
TC-UC02-4	UC-02: pedestrian request retained during clearance	Press side-0 button (1), then press it again during <code>FLASHING_DONT_WALK</code> .	The first request completes; a second <code>WALK</code> starts without a third press.	L1 terminal showed <code>DONT_WALK</code> for the first service followed by a second side-0 <code>WALK</code> .	Pass — runtime
TC-MSG-13	PA-11: override duration above the five-minute cap	With L1 operating normally, enter an override duration of 300001 ms at C1.	C1 rejects the value as <code>INVALID_DURATION</code> before sending a request to L1.	C1 displayed the duration rejection; L1 showed no override activation. This checks C1 validation, not an L1 <code>NACK</code> .	Pass — runtime
TC-MSG-15	UC-08 / CC-02: railway preemption takes priority over override	Put L1 in <code>RAILWAY_PREEMPTION</code> using RL1's train input; request a connector override from C1.	L1 rejects the conflicting request with <code>RAILWAY_CONFLICT</code> and remains preempted.	C1 showed <code>NACK reason=RAILWAY_CONFLICT</code> ; L1 did not enter <code>CENTRAL_OVERRIDE</code> .	Pass — runtime
TC-SC04B-2	RC-04: overlapping train occupancy windows	During RL1's first train-presence window, enter an approach from the other direction about 10 s later.	Gate opening waits until both occupancy windows expire.	RL1 terminal showed no gate-opening command when the first window expired; opening began after the second window ended.	Pass — runtime
TC-FAULT-01	RC-06: unconfirmed gate closure	At RL1, press <code>x</code> to fail the next gate confirmation, then <code>0</code> for a train approach.	RL1 enters a gate-confirmation fault and keeps train signals at <code>STOP</code> .	RL1 terminal showed the confirmation failure and fault; no <code>PROCEED</code> indication appeared.	Pass — runtime
TC-FAULT-22	RC-09/RC-10: railway fault recovery	After RL1 enters <code>FAULT</code> , press <code>x</code> to simulate confirmed-open gate feedback; send fault-clear with <code>f</code> at C1.	RL1 ACKs only after open confirmation, clears the fault, and returns to <code>OPEN</code> .	RL1 displayed confirmed-open feedback and returned to <code>OPEN</code> ; C1 displayed the fault-clear ACK.	Pass — runtime
TC-RACE-14	Concurrent train approaches from both directions	At RL1, enter <code>0</code> and <code>1</code> in quick succession while the crossing is initially <code>OPEN</code> .	Both train directions are retained; neither occupancy window overwrites the other.	RL1 terminal showed <code>PROCEED</code> for both directions after gate closure was confirmed.	Pass — runtime
TC-XNODE-11	RC-02: direct crossing-status delivery over Qnet	With C1, RL1, L1 and L2 running on their respective VMs, enter a train approach at RL1.	RL1's changed crossing status reaches both adjacent intersections and C1.	L1 and L2 showed railway preemption; C1's status view reflected RL1's crossing state.	Pass — runtime
TC-XNODE-13	PA-07: detect loss of one controller	With all nine local controllers available, stop L4's process.	C1 marks L4 <code>UNAVAILABLE</code> after its third missed heartbeat check; other controllers remain available.	C1 logged the three-miss transition and displayed L4 as <code>UNAVAILABLE</code> ; the other rows remained available.	Pass — runtime
TC-XNODE-15	PA-08: restore status after restart	Restart L4 after TC-XNODE-13, using the same Qnet mapping.	C1 marks L4 <code>AVAILABLE</code> when a valid heartbeat arrives; no restart of C1 is required.	L4 returned to <code>AVAILABLE</code> in C1's status view after heartbeat delivery.	Pass — runtime
TC-	PA-10: watchdog	Review <code>lx_watchdog.c</code>	Each watchdog checks	Code review found the separate	Pass —

FAULT-15	implementation	and <code>rlx_watchdog.c</code> ; no runtime fault injection.	its controller's progress counter and calls the local fault handler if progress stops.	watchdog threads, counter checks, and fault-handler calls. This does not demonstrate a runtime watchdog trip.	code review only
TC-TIME-05	PA-09: timing-profile offset boundary	Send offsets of 89999 and 90000 ms directly to L1.	89999 receives ACK; 90000 receives STALE_OR_UNSAFE_PROFILE NACK.	Not executed: C1's operator interface cannot enter arbitrary offsets and the required <code>test_client</code> was unavailable.	Skip
TC-FAULT-17	PA-10: Lx watchdog trip	Pause only L1's server thread while leaving its watchdog thread running.	The watchdog detects stalled phase-timer progress and puts L1 into <code>FAULT_SAFE</code> .	Not executed: the required per-thread debugger control was unavailable.	Skip

8.3 Timing and Real-Time Behavior

Table 16 separates configured timing values from observations. Lx and RLx print state changes without timestamps, so their intervals were checked using a manual stopwatch against the relevant controller terminal. C1's log timestamps have one-second resolution. These methods support approximate interval checks, but do not establish millisecond-level precision or worst-case response time.

Table 16. Representative timing results

Measurement	Method	Required / configured value	Observed result	Interpretation
Off-peak arterial green bounds	With L1 in <code>OFF_PEAK_SENSOR</code> , time <code>ARTERIAL_GREEN</code> → <code>ARTERIAL_YELLOW</code> on its terminal. For the minimum, assert connector demand (c) with no arterial demand. For the maximum, keep both arterial (a) and connector (c) demand asserted.	Minimum 8 s; maximum 40 s; exit conditions checked on the 100 ms phase tick and at 4 s extension points.	Approximately 8 s with only connector demand; approximately 40 s with both demands held.	Consistent with the configured bounds at stopwatch precision. With no connector demand, arterial green rests indefinitely; that condition cannot test either exit bound.
Pedestrian indication sequence	At a compatible Lx phase, press 1; time the terminal transitions <code>WALK</code> → <code>FLASHING_DONT_WALK</code> → <code>DONT_WALK</code> .	6 s <code>WALK</code> followed by 4 s flashing clearance; both are configured proof-of-concept values.	Approximately 6 s <code>WALK</code> and 4 s flashing clearance.	The observed order and approximate durations match the configured pedestrian sequence.
Railway warning to confirmed gate closure	On an RLx terminal, time <code>flashers_ON</code> after train input 0 to the confirmed-closed indication or the first permitted train <code>PROCEED</code> .	5 s warning before closing, plus 3 s simulated gate movement: approximately 8 s on the normal path.	Approximately 8–9 s.	Consistent with the normal crossing sequence. The 20 s fault deadline applies when closure is <i>not</i> confirmed; it is a different measurement.
Detection of a stopped local controller	Stop one available Lx process; observe C1's <code>UNAVAILABLE</code> transition and the "missed 3 consecutive heartbeats" entry in <code>central_log.txt</code> .	Heartbeat and C1 check periods: 1 s. C1 declares unavailability on the third missed check.	The transition was reported after the third missed check, approximately 3 s by manual observation.	Confirms the three-miss rule. Timer phases and the log's one-second timestamps do not support a claim of exactly 3.000 s from the stop command.
Central command response	Start a stopwatch on submission of the final C1 command parameter; stop when its <code>ACK</code> , <code>ACK_PENDING</code> , or <code>NACK</code> result appears in the C1 terminal.	PA-12 response target: within 1 s. <code>ACK_PENDING</code> acknowledges receipt; later activation is a separate interval.	Responses were reported as under 1 s at manual stopwatch precision.	Supports the response target for the observed trials. The method cannot establish a millisecond latency or a worst-case bound.
Maximum clear-route override duration	Request a 300000 ms override without pedestrian clearance or railway preemption. Start timing when C1 displays its <code>ACK</code> ; stop when Lx prints that the override expired.	Maximum accepted duration: 300 s, counted down on Lx's 100 ms phase tick.	Override expiry was observed at approximately 300 s.	Consistent with the configured five-minute cap at stopwatch precision. Boundary acceptance and rejection are separate protocol checks.

Measurement limits. Normal heartbeats are not individually timestamped in `central_log.txt`, so the display cannot independently establish an exact 1 Hz delivery rate. Cross-VM green-wave offsets also

depend on the VMs' wall clocks; assessing their numeric accuracy requires a recorded clock-skew check in addition to the phase observations.

8.4 Faults, Limitations, and Unresolved Results

No executed case in the reported run was marked Fail. **Pass does not imply complete coverage:** the skipped cases in **Section 8.1** and the implementation limits below remain open. **Table 17** records their impact, current handling, and the work needed to resolve them.

Table 17. Known issues and limitations

Issue / unresolved result	Impact	Current mitigation	Reproduction / evidence	Future correction
No Lx status message on every signal transition	The project brief requires an intersection status message whenever a light changes [1]. C1 currently learns the new state from the next successfully delivered heartbeat, so its view can lag a transition.	C1 receives the status summary in Lx's periodic heartbeat; Lx continues controlling its own signals.	<code>lx_comm.c</code> sends heartbeat summaries; no Lx transition path sends <code>MSG_STATUS</code> . This deviation is also recorded in Section 1.3.	Send <code>MSG_STATUS</code> when an Lx signal changes, then test delivery and C1 display updates.
Crossing status is not replayed to a late or reconnecting Lx	An adjacent Lx that misses a crossing-state change may retain an old view until RLx changes state again.	For the demonstration, bring all adjacent Lx controllers online before injecting a train approach and check connectivity at C1.	<code>TC-XNODE-12</code> ; <code>rlx_comm_broadcast_crossing_status_if_changed()</code> broadcasts on state change but has no resend or state-query path.	Send the current crossing state on peer reconnection or add an explicit state request and acknowledgement.
Single aggregate simulated gate state	The tests can check confirmation before train <code>PROCEED</code> , but cannot detect disagreement between two independent boom gates or position sensors described in the Initial Design Report [2].	Treat this as the documented proof-of-concept boundary; retain the confirmation-before- <code>PROCEED</code> rule.	Section 1.3; gate-confirmation fault tests such as <code>TC-FAULT-01</code> .	Model both gate and sensor pairs separately and add disagreement tests.
Stuck or silent sensors are not diagnosed	The keyboard inputs represent discrete events. A permanently silent train sensor, or a stuck train, vehicle, or pedestrian sensor, is not reliably detectable through this interface.	The test run exercises simulated events and gate faults; it does not claim sensor-health coverage.	<code>TC-FAULT-19/20/21</code> and the known sensor limits documented in the state-machine test plan.	Add persistent sensor states, diagnostic timeouts or self-tests, and corresponding fault-injection cases.
Raw IPC boundary tests need <code>test_client</code>	Twelve protocol and timing cases cannot be generated through C1's validated keyboard interface. Their intended ACK/NACK outcomes remain unverified at runtime.	All twelve are Skip, including <code>TC-TIME-05</code> ; they are excluded from runtime Pass results.	Test-plan environment (D); Skip breakdown in Section 8.1 .	Build a client that sends controlled <code>ipc_request_t</code> payloads and records the direct reply, then run the twelve cases.
Lx <code>FAULT_SAFE</code> and watchdog injection remain incompletely tested	Nine cases could not establish the required Lx fault state; three further cases require debugger control. The process-wide <code>kill -STOP</code> experiment was not recorded. Code review cannot prove runtime watchdog response.	Mark these cases Skip. The watchdog implementation has separate code-review coverage; RLx gate-fault and fault-clear paths have keyboard-driven runtime coverage.	<code>TC-FAULT-15/16/17/18</code> and the Skip breakdown in Section 8.1 .	Use controlled fault injection or a debugger that pauses only the server thread; record the <code>kill -STOP</code> experiment separately and rerun dependent cases.
Cross-VM clock skew is not controlled by the application	Green-wave offsets use each Lx VM's wall clock. Clock differences can shift the observed progression even when the assigned offsets are correct.	Section 8.3 reports local timing measurements without claiming a measured cross-VM offset accuracy.	<code>lx_fsm_apply_off_set_locked()</code> reads <code>CLOCK_REALTIME</code> ; the timing plan requires a clock-skew check before numeric	Synchronise VM clocks and record their skew alongside measured L1–L3–L5 and L2–L4–L6 offsets.

			offset tests.	
Blocking synchronous send has no timeout	If a destination process remains present but stops receiving or replying, the dedicated IPC client thread can remain blocked in <code>MsgSend()</code> . Subsequent outbound messages wait in the 16-entry queue, and heartbeat attempts may not be complete; therefore, the expected three-miss transition to <code>LINK_DEGRADED_LOCAL</code> can be delayed or may not occur while the send remains blocked.	The IPC server, local FSM timers, sensor thread, and watchdog run independently of the blocked client thread. Local signal or crossing operation can therefore continue, subject to the outgoing queue eventually becoming full.	<code>ipc_client_thread_main()</code> in the replicated <code>src/qnet_utils.c</code> performs blocking <code>MsgSend()</code> without configuring a send or reply timeout. The unexecuted <code>kill -STOP / kill -CONT</code> case is recorded in the Section 8.1 Skip breakdown.	Configure QNX send/reply timeouts or replace the blocking send path with a bounded asynchronous or retry design. Execute and record the <code>kill -STOP / kill -CONT</code> recovery case.

9. Conclusion

The implementation demonstrates a distributed QNX traffic-control proof of concept deployed as **ten controller instances** across *Qnet*. **Six intersection controllers** maintain autonomous vehicle and pedestrian sequencing, while **three railway controllers** enforce local crossing protection and send crossing state directly to adjacent intersections without depending on **C1** as a relay. **C1** provides monitoring, logging, scheduling, and validated supervisory requests without directly controlling local outputs. The reported verification run covered normal operation, timing boundaries, railway preemption, conflicting requests, faults, and recovery, with **201 runtime passes, six code-review passes, 25 skipped cases, and no recorded failures**. Material limitations remain in the simulated hardware model, unexecuted protocol and fault-injection cases, unreplayed crossing-state updates, and the blocking `MsgSend()` path without a configured timeout. Within these boundaries, the implementation demonstrates the required local autonomy and core railway-preemption safety behavior.

References

- [1] RMIT University, "RTS Final Project," RMIT Canvas. Accessed: Sep. 19, 2026. [Online]. Available: <https://rmit.instructure.com/courses/171520/files/55415560?wrap=1>
- [2] D. N. Tran, H. Le, and M. T. Hoang, "Initial Design Report," RMIT University Vietnam, Ho Chi Minh City, Vietnam, project report, 2026.